

HRNN: A Hybrid Graph Index for Approximate Reverse k -Nearest Neighbor Search on High-Dimensional Vectors

Wenxuan Xia

HKUST (GZ)

China

wxia248@connect.hkust-gz.edu.cn

Wentao Li

University of Leicester

United Kingdom

wl226@leicester.ac.uk

Mingyu Yang

HKUST (GZ) & HKUST

China

myang250@connect.hkust-gz.edu.cn

Wei Wang

HKUST (GZ) & HKUST

China

weiwcs@ust.hk

ABSTRACT

Reverse k -nearest neighbor (RkNN) search returns all data points that regard a query vector as one of their k -nearest neighbors (kNNs). Existing RkNN methods typically follow a **filter-and-verification framework**: vectors near the query vector are first collected as candidates and then verified against their k NN-radius (i.e., the distance to their k -th nearest neighbor). However, existing methods face two key limitations in high-dimensional spaces. First, nearby vectors often do not belong to the query’s true RkNN set, resulting in excessive candidate expansion overhead. Second, existing methods compute k NN-radius online during verification, incurring substantial query-processing cost.

To address these limitations, we propose HRNN, a hybrid graph index for approximate RkNN search. (1) Rather than directly treating nearby vectors as RkNN candidates, HRNN uses them as proxy points based on the **assumption** that a query’s RkNN results can often be discovered through the RkNN results of its nearby vectors. (2) To reduce verification cost, HRNN **materializes** high-fidelity k NN-radius offline, eliminating expensive online reconstruction while preserving accuracy. HRNN combines a navigation graph, a ranked KNN graph, and reverse-neighbor lists into a hybrid index that supports efficient proxy retrieval, candidate generation, and k NN-radius access. We also develop efficient index construction and append-only maintenance algorithms. Extensive experiments show that HRNN consistently outperforms existing methods, achieving up to one order of magnitude higher throughput. Moreover, HRNN scales to datasets containing up to 10 million high-dimensional vectors while supporting efficient dynamic index maintenance.

PVLDB Reference Format:

Wenxuan Xia, Mingyu Yang, Wentao Li, and Wei Wang. HRNN: A Hybrid Graph Index for Approximate Reverse k -Nearest Neighbor Search on High-Dimensional Vectors. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/gegeji/hrnn>.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

With the rapid development of large language models (LLMs) and intelligent agents [27, 46], modern data are increasingly represented as high-dimensional vectors, such as text embeddings, image features, and other multi-modal representations [2, 58, 59]. Given a dataset D of vectors/data points, the **k -nearest neighbors (kNN)** of a data point are the k points closest to it, where distance is typically measured using Euclidean distance. Conversely, the **reverse k -nearest neighbors (RkNN)** of a query point q consist of all data points in D that regard q as one of their k -nearest neighbors. Given a dataset D , the RkNN search problem aims to efficiently retrieve the RkNN set of a query vector q from D .

The RkNN set of a query vector q naturally reflects its influence over the dataset D , since each vector in the RkNN set regards q as one of its k -nearest neighbors. Consequently, RkNN search has been widely used in applications that require influence analysis and reverse relationship discovery. For example, in spatial analytics, RkNN search can identify attractive facility locations by finding sites that appear among the k -nearest neighbors of a large number of customers [7]. In retrieval-augmented generation (RAG) systems, RkNN search can help identify influential knowledge chunks that are frequently referenced by many queries. More broadly, RkNN search has been extensively studied in machine learning tasks that rely on reverse influence relationships among data points, including density-based clustering [5, 10, 20], outlier detection [4, 19, 37], and class-imbalance sampling [38, 60].

To process the RkNN search for a query q , an **intuitive** approach is to first compute the k NN set of every point o in the dataset D and obtain its **k NN-radius**, i.e., the distance between o and its k -th nearest neighbor. Then, by definition, if the distance between q and point $o \in D$ does not exceed the k NN radius of o , o is identified as an RkNN of q . However, this intuitive approach becomes prohibitively expensive when the dataset is large or the vector dimensionality is high. Although prior studies [7, 40] have explored spatial indexes such as R-trees to accelerate RkNN search, exact RkNN retrieval in high-dimensional vector spaces remains challenging. Thus, this paper focuses on approximate RkNN (i.e., ARkNN) search.

Existing Solutions. Existing methods for ARkNN search generally follow a **filter-and-verification framework**. They first retrieve a superset of potential RkNN results for a query q as candidates, and then verify each candidate using a process similar to the intuitive

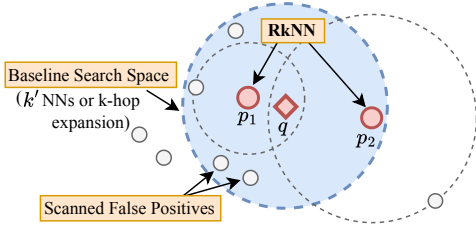


Figure 1: Illustration of the inefficiency of existing methods. The radius of the circles centered at points (e.g., p_1 and p_2) in the dataset D denotes their k NN-radius. Given a query vector q , the goal is to retrieve its RkNN results (p_1 and p_2 in this example, as we set $k = 1$). Existing methods typically explore nearby neighbors, such as through top- k' ($k' \gg k$) nearest-neighbor search or k -hop graph expansion, and then verify whether the retrieved candidates are true RkNN results. However, to reach p_2 , which has a large k NN-radius due to its sparse neighborhood, these methods must traverse a large search region and examine many false positive candidates (gray points), resulting in substantial unnecessary computation.

approach described above. Early methods mainly rely on spatial partitioning [42, 43] or precomputed distance bounds [1] to prune unpromising vectors from the dataset. Yet, these approaches suffer from high preprocessing and indexing costs in high-dimensional spaces due to the curse of dimensionality [21]. More recent methods, including SFT [39], RDT [6], and the state-of-the-art HAMG [41], are based on the assumption that **the RkNNs of a query vector often overlap significantly with its nearby neighbors**. Thus, these methods first retrieve nearby neighbors as candidates, such as the top- k' nearest neighbors of the query vector in SFT [39] and RDT [6], where $k' \gg k$, or the k -hop neighbors in graph-based methods such as HAMG [41]. They then verify whether each candidate belongs to the RkNN set of the query by comparing the query distance against the candidate’s k NN-radius.

Limitations. Although recent methods achieve good performance by considering the properties of high-dimensional spaces, they still suffer from inherent limitations that restrict their efficiency.

Limitation 1: The mismatch between RkNN and nearby neighbors.

Existing methods need to examine a large number of nearby neighbors to achieve high recall, because true RkNN results can be far away from the query vector in the embedding space (see Fig. 1 and Fig. 4). For example, a point $o \in D$ may still regard query vector q as one of its k -nearest neighbors if o lies in a sparse region and therefore has a large k NN-radius. As a result, the query must search a large neighborhood region before reaching o , leading to excessive candidate expansion and unnecessary verification cost. This mismatch between RkNN results and nearby neighbors fundamentally limits the efficiency of existing methods.

Limitation 2: High verification cost. After generating candidate points, existing methods verify whether a candidate o belongs to the final RkNN result by comparing the distance between q and o against the k NN-radius of o . Specifically, if $\delta(q, o)$ is no larger than the k NN-radius of o , then q is regarded as one of the k -nearest neighbors of o , and thus o is an RkNN of q . Yet, the k NN-radius of each candidate is typically computed online during query time [6, 41, 54], as precomputing and storing these distances for all k values incurs prohibitive preprocessing costs or requires rebuilding the index.

Table 1: Comparison between HRNN and existing RkNN methods.

	HRNN (Ours)	HAMG	RDT	SFT
Filter (Candidate Generation)	Reverse- neighbor lists of proxies	k -hop graph traversal	Incremental k NN search	One-shot k NN search
Verification	Pre- materialized k NN-radius	Online computation	Online computation	Online computation

As a result, candidate verification introduces substantial computational overhead and becomes a major bottleneck in existing RkNN methods, as confirmed by our empirical studies.

Our Solution. To address the limitations of existing methods, we propose HRNN, a **Hybrid** graph index for approximate **Reverse** k -**Nearest Neighbor** search. (1) To overcome the mismatch between RkNN results and nearby neighbors, rather than assuming that a query’s nearby neighbors directly contain its RkNN results, we make a different assumption: **a query’s RkNN results can often be discovered through the RkNN results of its nearby neighbors**. Based on this assumption, HRNN first retrieves the query’s nearby neighbors (i.e., k NNs) as **proxy** points and then leverages their RkNN results to recover the query’s RkNN results. To support this process, each point maintains a **reverse-neighbor list** that records the points in D whose nearest-neighbor lists contain it, enabling efficient retrieval of its RkNN results for arbitrary k when used as a proxy. This design avoids excessive neighborhood expansion while maintaining high retrieval accuracy. (2) To address the high verification cost of existing methods, HRNN eliminates the expensive online computation of k NN-radius during query processing. Instead, it materializes k NN-radius information offline, enabling efficient candidate verification at query time.

Achieving these goals is non-trivial, as the index must simultaneously support k NN retrieval for query vectors, reverse-neighbor-list access for proxy points, and fast k NN-radius lookup for arbitrary values of k . To this end, HRNN integrates two complementary graph structures. First, it employs a navigation graph, such as HNSW [33], to retrieve the query’s nearest neighbors as proxy points. Second, it maintains a ranked KNN graph [12] that stores nearest-neighbor relationships among data points. This graph provides direct access to pre-materialized k NN-radius for verification and serves as the basis for constructing reverse-neighbor lists, which enable efficient retrieval of proxy points’ RkNN results for candidate generation. By tightly integrating a navigation graph, a ranked KNN graph, and reverse-neighbor lists into a unified index, HRNN enables efficient RkNN query processing. We further develop efficient index construction and insertion-only maintenance algorithms. A comparison between HRNN and existing methods is in Table 1.

Contributions. We summarize our main contributions as follows.

Problem analysis (§3). We identify two key shortcomings of existing methods for high-dimensional ARkNN search. First, true RkNN results can be far away from the query vector in the embedding space, and thus the mismatch between RkNN results and nearby neighbors leads to excessive false positive candidates and unnecessary computation. Second, the k NN-radius of each candidate is computed online, which incurs huge overhead during the verification stage.

Table 2: Notation summary.

Notation	Description
G_{HNSW}	HNSW navigation graph
G_{KNN}	Ranked KNN graph
$G_{\text{KNN}}[o]$	Ranked KNN list of o , sorted by distance
$R[o]$	Reverse-neighbor list of o
$r_k(o)$	k NN radius (distance to the k -th nearest neighbor) of o
k	Target parameter for k NN or Rk NN search
K	Number of stored neighbors per vertex in G_{KNN}
Θ	Rank threshold for reverse candidate generation
m	Number of proxy vectors retrieved from G_{HNSW}
Θ_u	Rank threshold used during insertion maintenance
m_u	Number of update proxies used during insertion maintenance

Hybrid index design (§4). We propose HRNN, a hybrid graph index for efficient ARkNN search. HRNN integrates a navigation graph, a ranked KNN graph, and reverse-neighbor lists derived from the ranked KNN graph. The navigation graph is used to efficiently retrieve nearby proxy points, while the reverse-neighbor lists are used to recover the RkNN candidates of the query vector. In addition, the ranked KNN graph materializes k NN-radius information for arbitrary values of k , enabling efficient candidate verification without expensive online k NN-radius computation.

Index construction and maintenance (§4). We develop efficient algorithms for HRNN index construction and maintenance. Specifically, we use the navigation graph as initialization seeds to improve the construction quality of the ranked KNN graph. We further propose efficient methods to construct reverse-neighbor lists and support insertion-only index maintenance under dynamic updates.

Extensive experimental studies (§5). Extensive experiments on four real-world datasets show that HRNN improves the recall-throughput trade-off over strong RkNN baselines by up to one order of magnitude. Additional ablation, varying- k , and scalability experiments further validate the effectiveness of our HRNN.

2 PRELIMINARY

We first formulate the approximate RkNN search problem, and then introduce the graph-based index that supports k NN retrieval.

2.1 Problem Definition

We first introduce the definitions of k NN and RkNN, and then formally define the studied problem. Table 2 summarizes the commonly used notations in this paper.

Definition 2.1 (k -Nearest Neighbor (k NN)). Given a dataset D of points in Euclidean space $\mathbb{R}^d$, a d -dimensional query vector q , and an integer k , the k NN set $N_k(q)$ of q is a subset of D such that $|N_k(q)| = k$ and $\forall v \in N_k(q), \forall x \in D \setminus N_k(q), \delta(v, q) \leq \delta(x, q)$, where $\delta(v, q)$ denotes the Euclidean distance between v and q .

Note that q may not belong to D . For each point $o \in D$, let $N_k(o)$ denote its k NN in D , and let its **k NN-radius** be defined as $r_k(o) = \delta(o, v)$, where $v \in N_k(o)$ is the k -th nearest neighbor of o .

Definition 2.2 (Reverse k -Nearest Neighbor (Rk NN)). Given a dataset $D \subseteq \mathbb{R}^d$, a d -dimensional query vector q , and an integer k , the RkNN set of q is defined as $A_k(q) = \{o \in D \mid \delta(q, o) \leq r_k(o)\}$.

The RkNN set $A_k(q)$ contains all points $o \in D$ that regard q as one of their k nearest neighbors. Equivalently, the distance between q and o is no larger than the k NN-radius of point o . Since exact RkNN search is computationally expensive in high-dimensional

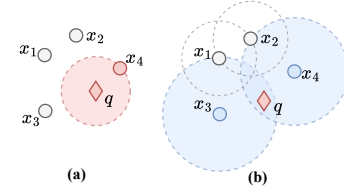


Figure 2: Illustrative examples of k NN and RkNN search.

spaces, existing methods typically trade accuracy for efficiency by returning approximate results. We therefore formally define the problem of **approximate RkNN (ARkNN) search** as follows.

Definition 2.3 (Approximate Reverse k -Nearest Neighbor (ARkNN) Search). Given a dataset D , a query vector q , and an integer k , an ARkNN search returns a set $\hat{A}_k(q) \subseteq D$ as an approximation to the exact RkNN result set $A_k(q)$. Its accuracy is evaluated against $A_k(q)$ rather than by an ϵ -relaxed membership predicate.

Following [41], we use $\text{Recall}@k$ to evaluate the accuracy of ARkNN results against the exact RkNN ground truth.

Definition 2.4 (Recall@ k). For a query q , let $A_k(q)$ denote the exact RkNN result set and let $\hat{A}_k(q)$ denote the approximate result set. The per-query $\text{Recall}@k$ is defined as

$$\text{Recall}@k(q) = \begin{cases} \frac{|A_k(q) \cap \hat{A}_k(q)|}{|\hat{A}_k(q)|}, & |A_k(q)| > 0, \\ 1, & |A_k(q)| = 0 \text{ and } |\hat{A}_k(q)| = 0, \\ 0, & |A_k(q)| = 0 \text{ and } |\hat{A}_k(q)| > 0. \end{cases}$$

For a query workload Q , $\text{Recall}@k$ is computed as the average of $\text{Recall}@k(q)$ over all $q \in Q$.

Example 2.5. Fig. 2 shows k NN and RkNN search. Let $x_1, \dots, x_4$ be data points and q the query, with $k = 1$. In Fig. 2(a), the nearest neighbor of q is x_4 , and the (radius of the) circle centered at q denotes its k NN-radius. In Fig. 2(b), the circles centered at $x_1, \dots, x_4$ indicate their k NN-radius. Since q lies within the k NN-radius of x_3 and x_4 but outside those of x_1 and x_2 , the RkNN result set of q is $\{x_3, x_4\}$.

2.2 Graph-Based k NN Search

We briefly introduce graph-based methods for high-dimensional k NN search, which serve as the foundation of the studied ARkNN search problem. Among existing high-dimensional k NN methods, graph-based approaches [8, 12–14, 18, 22, 23, 28, 32, 33, 36, 47, 49, 50] are particularly popular due to their state-of-the-art performance. These methods model the points in dataset D as vertices in a graph and connect nearby vertices through directed or undirected edges. As a result, k NN search for a query vector q becomes a graph traversal process over the constructed index.

Ranked KNN Graph. We first introduce the ranked KNN graph, where K is a user-defined parameter chosen to be sufficiently large to support k NN search for arbitrary $k \leq K$.

Definition 2.6 (Ranked KNN Graph). Given a dataset $D \subseteq \mathbb{R}^d$ and an integer K , the ranked KNN graph $G_{\text{KNN}} = (V, E)$ is a directed graph with vertex set $V = D$. Each vertex $o \in V$ stores an ordered **KNN list** of its K nearest neighbors and their distances: $G_{\text{KNN}}[o] = \{(v_1, \delta(o, v_1)), \dots, (v_K, \delta(o, v_K))\}$, where $\delta(o, v_1) \leq \dots \leq \delta(o, v_K)$. A directed edge $o \rightarrow v_j$ exists if v_j is the j -th nearest neighbor of o , i.e., $G_{\text{KNN}}[o, j] = v_j$. For simplicity, we write $G_{\text{KNN}}[o, j] = v_j$, although the actual entry is $(v_j, \delta(o, v_j))$.

Algorithm 1: Ranked KNN Graph Construction

Input: Dataset D , integer K
Output: Ranked KNN graph G_{KNN}

```
1 Initialize  $G_{KNN}[o]$  with  $K$  random points for each  $o \in D$ ;  
2 repeat  
3   Create  $R[o] \leftarrow \{x \mid o \in G_{KNN}[x]\}$  for each  $o \in D$ ;  
4   for each  $o \in D$  do  $N[o] \leftarrow G_{KNN}[o] \cup R[o]$ ;  
5   for each  $o \in D$  do  
6     for each neighbor pair  $(u, v)$  in  $N[o]$  do  
7       Compute  $d = \delta(u, v)$ ;  
8       Insert  $(v, d)$  into  $G_{KNN}[u]$  if improved;  
9       Insert  $(u, d)$  into  $G_{KNN}[v]$  if improved;  
10 until no neighbor list changes;
```

The ranked G_{KNN} captures nearest-neighbor relationships from the perspective of each point in D . Transposing these relationships yields reverse-neighbor lists, which record, for each vector, the points that regard it as one of their top- K nearest neighbors. Consequently, the RkNN results of any dataset vector can be obtained directly from its reverse-neighbor list for any $k \leq K$.

Definition 2.7 (Reverse-Neighbor List). Given a ranked G_{KNN} , the reverse-neighbor list of a vector $o \in D$, denoted by $R[o]$, is defined as $R[o] = \{(v, j) \mid v \in D, G_{KNN}[v, j] = o, 1 \leq j \leq K\}$, where (v, j) indicates that o is the j -th nearest neighbor of v .

Index construction. Exact construction of ranked KNN graph requires $O(|D|^2)$ distance computations, since the K -nearest neighbors must be identified for every point in D . To improve efficiency, NNDescent [12] iteratively refines the KNN list of each point based on the neighbors of its current neighbors. Algorithm 1 shows how to create the ranked KNN graph using NNDescent. It first initializes the neighbor list $G_{KNN}[o]$ of each point $o \in D$ with K random points (Line 1). During each iteration, it constructs a reverse-neighbor list $R[o]$ for every point o , which contains all points that currently include o in their neighbor lists (Line 3). The forward and reverse neighbor lists are then merged to form the neighbor set $N[o]$ (Line 4). For each neighbor pair (u, v) in $N[o]$ (Lines 5–6), the algorithm computes their distance $\delta(u, v)$ and attempts to insert (v, d) into $G_{KNN}[u]$ and (u, d) into $G_{KNN}[v]$ if the new neighbor improves the current ranked neighbor list (Lines 7–9). Here, we say that (v, d) updates $G_{KNN}[u]$ if the distance $d = \delta(u, v)$ is smaller than the distance between u and at least one neighbor currently stored in $G_{KNN}[u]$. The refinement process repeats until no neighbor list changes further (Line 10). For simplicity, we omit several optimizations, such as neighbor sampling and lazy updates.

Navigation Graph. Ranked KNN graphs mainly connect each point to its local neighbors and therefore lack long-range edges. As a result, greedy graph traversal from a random entry point can easily become trapped in local optima [18]. To address this limitation, navigation graphs [14], such as the Hierarchical Navigable Small World (HNSW) graph [33], have been proposed.

Definition 2.8 (HNSW Graph). Given a dataset $D \subset \mathbb{R}^d$, an HNSW graph is an L -layer directed proximity graph $G_{HNSW} = \{G^{(0)}, G^{(1)}, \dots, G^{(L)}\}$, where each layer $G^{(\ell)} = (V^{(\ell)}, E^{(\ell)})$ is defined over a subset $V^{(\ell)} \subseteq D$, satisfying $V^{(L)} \subseteq V^{(L-1)} \subseteq \dots \subseteq V^{(0)} = D$. Each point $o \in D$ is assigned a maximum level $\lambda(o)$ and

Algorithm 2: GRAPH-SEARCH(G, q, k, ep, ef)

Input: Graph G , query vector q , target number of results k , entry point ep , beam width ef
Output: Candidate set $W[q]$ of size k

```
1 Initialize min-heap  $Q \leftarrow \{ep\}$  ordered by  $\delta(q, \cdot)$ ;  
2 Initialize max-heap  $W[q] \leftarrow \{ep\}$  ordered by  $\delta(q, \cdot)$ ;  
3 Mark  $ep$  as visited;  
4 while  $Q \neq \emptyset$  do  
5   Extract the closest vertex  $v$  from  $Q$ ;  
6   Let  $u$  be the farthest vertex in  $W[q]$ ;  
7   if  $\delta(q, v) > \delta(q, u)$  then break;  
8   for each unvisited neighbor  $o \in G[v]$  do  
9     Mark  $o$  as visited;  
10    if  $|W[q]| < k$  or  $\delta(q, o) < \delta(q, u)$  then  
11      Insert  $o$  into both  $Q$  and  $W[q]$ ;  
12      if  $|W[q]| > k$  then  
13        Remove the farthest vertex from  $W[q]$ ;  
14      if  $|Q| > ef$  then  
15        Remove the farthest vertex from  $Q$ ;  
16 return  $W[q]$ ;
```

appears in all layers $G^{(0)}, \dots, G^{(\lambda(o))}$. At each layer $\ell \leq \lambda(o)$, point o maintains a bounded sized neighbor list $G^{(\ell)}[o] = \{v_1, \dots, v_M\}$, where each $v_i \in V^{(\ell)}$ is selected using proximity-based pruning heuristics. A directed edge $o \rightarrow v_i$ exists in layer $G^{(\ell)}$ if $v_i \in G^{(\ell)}[o]$. *Search on the HNSW graph.* Given a query q , an HNSW graph G_{HNSW} , the target result size k , and a beam width ef , search on the HNSW index proceeds from the top layer to the bottom layer. Starting from the global entry point in the top layer, the algorithm first performs greedy search on each upper layer with beam width $ef = 1$, so that only the closest candidate is retained and used as the entry point for the next lower layer. After reaching the bottom layer $G^{(0)}$, the algorithm performs beam search with a larger beam width $ef > 1$ to retrieve the final k NN result for the query vector q .

The search within each layer $G^{(\ell)}$ follows Algorithm 2. Given the current entry point ep , the algorithm initializes a candidate heap Q and a result heap $W[q]$ with ep (Lines 1–2), and marks it as visited (Line 3). It then repeatedly extracts the closest vertex v from Q (Line 5) and compares it with the farthest vertex u in $W[q]$ (Line 6). If v is farther from q than u , the search stops because no remaining candidate can improve the current result (Line 7). Otherwise, the algorithm scans each unvisited neighbor $o \in G^{(\ell)}[v]$ (Line 8), marks it as visited (Line 9), and inserts it into both Q and $W[q]$. If $W[q]$ has fewer than k vertices or o is closer to q than the current farthest candidate (Lines 10–11). If $W[q]$ or Q exceeds their size limits, the farthest vertex is removed (Line 12–15). The search returns $W[q]$ as the result (Line 16).

Example 2.9. Fig. 3(a) illustrates an HNSW graph with three layers, where $G^{(2)}$ is the top layer and $G^{(0)}$ is the bottom layer. Given a query vector q , the top-down routing phase starts from the global entry point $ep = x_4$ and performs greedy search ($ef = 1$) on each layer. The search progressively descends through the hierarchy until reaching the bottom layer, where x_7 is identified as the entry point. Fig. 3(b) illustrates the bottom-layer beam search ($ef = 2$). Starting from x_7 , the search visits x_5 and inserts x_2 and x_9 into the candidate queue. Since $\delta(q, x_2) < \delta(q, x_9)$, x_2 is expanded

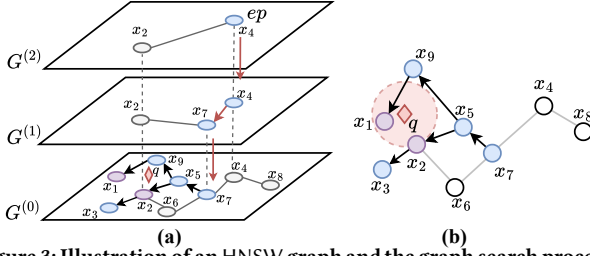


Figure 3: Illustration of an HNSW graph and the graph search process. first, followed by x_9 , which discovers x_1 . The final 2NN result is $W[q] = x_1, x_2$ (shaded in purple).

3 PROBLEM ANALYSIS

We summarize existing methods for RkNN search in Section 3.1, and then analyze their two limitations in Section 3.2. This analysis motivates our method presented in Section 4. For simplicity, we use the term RkNN search to refer to approximate RkNN search throughout the remainder of the paper unless otherwise specified.

3.1 Existing Solutions

The main challenge in answering the RkNN search problem lies in two aspects. First, unlike kNN search, where the result vectors are directly close to the query vector q , the vectors in the RkNN set do not necessarily have explicit proximity relationships with q . Specifically, RkNN search is defined from the perspective of dataset points $o \in D$: we need to determine whether q belongs to the kNN set of points o by checking the kNN-radius of o . Second, unlike kNN search, which always returns exactly k vectors, the number of reverse kNN results for a query vector is unknown in advance.

To process the RkNN search problem for a query vector q over dataset D , an intuitive approach is to directly follow the definition of RkNN. Specifically, the algorithm scans all points $o \in D$ and checks whether the distance $\delta(q, o)$ is no larger than the kNN-radius $r_k(o)$ of o , where $r_k(o)$ is defined as the distance from o to its k -th nearest neighbor. If so, o is included in the RkNN set of q . However, this approach is computationally expensive for large-scale datasets.

Filter-and-Verification Framework. To improve efficiency, existing methods typically adopt a filter-and-verification framework. Instead of examining all points in D , these methods first reduce the search space to a candidate set $C \subseteq D$, and then verify whether each candidate point belongs to the RkNN result set of the query vector q . To obtain a compact candidate set C , existing methods commonly assume that **data points close to q are more likely to regard q as one of their k nearest neighbors, and thus are more likely to belong to the RkNN result set of q** . Based on this assumption, methods such as SFT [39] and RDT [6] first perform nearest-neighbor search and then examine nearby neighbors as candidates until a stopping condition is satisfied.

The State-of-the-Art. More recently, HAMG [41] was proposed following the same intuition. A key insight of HAMG is that, on the Monotonic Relative Neighborhood (MRN) graph [3, 8, 12–14, 18, 22, 23, 28, 30, 32, 33, 36, 45, 47, 49, 50, 53], the RkNN results of a query vector q are guaranteed to lie within the k -hop neighborhood of q . Based on this property, HAMG treats these nearby graph neighbors as the candidate set C . Specifically, HAMG adopts the hierarchical structure of the (HNSW) graph in Definition 2.8,

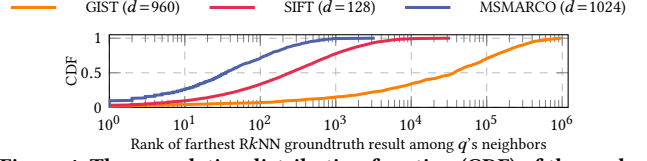


Figure 4: The cumulative distribution function (CDF) of the ranks of points in the RkNN result sets of query vectors. A higher rank indicates that more nearby neighbors must be explored before the exact RkNN ($k = 10$) results can be retrieved.

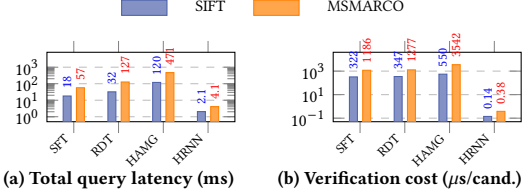


Figure 5: Comparison of query performance for approximate RkNN search with $k = 10$ at Recall@10 = 0.99. (a) shows the total query latency of HRNN and the baselines (SFT, RDT, and HAMG). (b) reports the per-candidate verification cost of HRNN and the baselines.

while modifying the bottom layer $G^{(0)}$ to better approximate the MRN graph structure. Using this adapted (HNSW) graph, HAMG retrieves the k -hop neighbors of q as candidates and then verifies whether $\delta(q, o) \leq r_k(o)$ for each candidate $o \in C$ to determine if o belongs to the RkNN set of q . More details can be found in [41].

3.2 Limitation Analysis

Existing methods avoid scanning the entire dataset D to retrieve the RkNN results of a query vector q . Yet, they still suffer from two major limitations that reduce their efficiency on large datasets.

Limitation 1: The mismatch between RkNN and nearby neighbors. Existing methods rely on the assumption that nearby neighbors of q (e.g., nearest neighbors in SFT and RDT, or graph neighbors within k hops in HAMG) contain all RkNN results. Although this assumption generally holds, it can still lead to high query latency in practice. As illustrated in the introduction (Fig. 1), when an object $o \in D$ has a large kNN-radius, the query vector q may still belong to the kNN set of o even if o is far away from q . Consequently, the search algorithm must traverse a large number of nearby neighbors before reaching o . Even for the state-of-the-art HAMG, the number of k -hop graph neighbors can become extremely large due to the graph expansion effect. As a result, a substantial amount of computation spent exploring nearby neighbors is unnecessary.

To further validate this limitation, we report the statistics in Fig. 4. We evaluate three datasets, GIST, SIFT, and MSMARCO, with dimensions 960, 128, and 1024, respectively, and set $k = 10$ for RkNN search. For each query q , we compute its exact RkNN result set. For every object in the result set, we measure its rank in the nearest-neighbor ordering of q , where the closest neighbor has rank 1 and the farthest point has rank $|D|$. We then compute the cumulative distribution function (CDF) of these rankings. The results show that all three datasets require exploring a large number of nearby neighbors to retrieve all RkNN results. For example, on SIFT, more than 50% of queries require scanning over 10^3 nearby neighbors to retrieve all RkNN results. More critically, some queries

require exploring an extremely large neighborhood region. On GIST, certain queries need to scan more than 10^6 nearby neighbors before all RkNN results can be identified. These observations indicate that relying solely on nearby neighbors is inefficient for RkNN search.

Limitation 2: High Verification Cost. The first limitation implies that existing RkNN methods often explore a large number of unnecessary neighbors during the filter stage, producing a candidate set C that is substantially larger than the true RkNN result set. Then, for each candidate vector $o \in C$, the algorithm must determine whether $\delta(o, q) \leq r_k(o)$. If the condition does not hold, then q is not among the k nearest neighbors of o , and o is discarded; otherwise, o is reported as an RkNN result. Consequently, efficient computation of the k NN-radius $r_k(o)$ of o is critical to RkNN search performance. Obtaining this value requires issuing a k NN query centered at o , as precomputing k NN-radius and supporting queries for arbitrary k values necessitate either prohibitive preprocessing cost or even complete index reconstruction in existing indexes [6, 39, 41, 54]. Besides, since the candidate set C is not known in advance and can be very large in practice, existing methods incur substantial overhead by computing k NN-radius online during query processing.

To quantify this limitation, we conduct experiments on the real-world SIFT and MSMARCO datasets. We compare the runtime of approximate RkNN search across HRNN and the baseline methods (SFT, RDT, and HAMG), with the results reported in Fig. 5. As shown in Fig. 5(a), HRNN achieves up to one order of magnitude lower query latency than the baselines. Fig. 5(b) further reveals the underlying reason: existing methods spend significantly more time verifying each candidate because they must compute the k NN-radius online. These results demonstrate that online k NN-radius computation is a major bottleneck for existing RkNN methods.

4 THE PROPOSED METHOD

We first introduce the proposed index structure in Section 4.1, followed by the query processing algorithm in Section 4.2. Then, Section 4.3 describes the index construction procedure, and Section 4.4 presents the append-only index maintenance method.

4.1 Index Structure

We now present the key ideas for addressing the two limitations of existing RkNN methods.

Idea 1. To address Limitation 1, we revisit the relationship between k NN and RkNN search. Existing methods often assume that nearby vectors tend to share common properties, and therefore treat the nearest neighbors of the query as candidates for RkNN search. However, as shown in our analysis, nearby neighbors are often not true RkNN results. Instead, we make a different assumption: **a query’s RkNN results can often be discovered through the RkNN results of its nearby neighbors.** This intuition is empirically validated in Fig. 6.

This assumption follows from spatial proximity: as q and its nearby neighbors lie close to each other, they are likely to fall within the k NN radius of the same point x . Thus, if x is an RkNN of q ’s nearby neighbors, it is also likely to appear in the RkNN sets of q . Based on this intuition, we first retrieve a set of nearby vectors as *proxies*. Rather than treating these proxies as RkNN candidates, we leverage their reverse-neighbor lists, which provide their RkNN results for arbitrary k , to generate candidates for the query. This

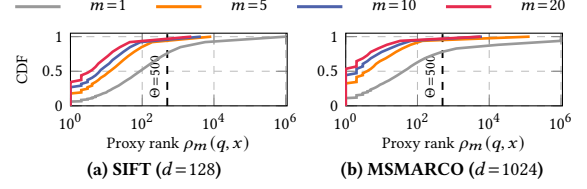


Figure 6: To validate our assumption, we evaluate the proxy ranks of exact RkNN results on the SIFT and MSMARCO datasets. For each query vector q , its top- m nearest neighbors are used as proxies ($m \in \{1, 5, 10, 20\}$). Consider an exact RkNN result x ($k = 10$) of q . We define the proxy rank of x as the smallest rank position of any proxy of q in x ’s ranked KNN list. By Definition 2.7, if a proxy appears at rank r in x ’s ranked KNN list, then x appears in the proxy’s reverse-neighbor list and can be discovered after scanning its reverse-neighbor list. Therefore, a small proxy rank of r indicates that x can be recovered through at least one proxy after examining only the first few entries of its reverse-neighbor list. The results show that proxy ranks are generally small and decrease as m increases. In particular, about 50% of the RkNN results have proxy ranks below 50, indicating that most results can be recovered by scanning only a short prefix of the proxies’ reverse-neighbor lists.

design avoids excessive neighborhood expansion while keeping high recall.

Idea 2. To address Limitation 2, we eliminate the expensive online computation of k NN-radius during verification. Recall that the k NN-radius of a vector $o \in D$ is simply the distance between o and its k -th nearest neighbor. Thus, if we materialize a ranked neighbor list for each vector and store the corresponding distances in ascending order, the k NN-radius for any value of k can be retrieved directly through a lookup. This transforms verification from an expensive online search into a lightweight index-access operation.

Index Design. To realize the above intuitions, the index must support three operations: (1) efficiently retrieving nearby neighbors of a query vector as proxies, (2) obtaining the reverse-neighbor lists of proxy points to generate RkNN candidates for the query, and (3) maintaining ranked neighbor lists for all points in the dataset so that k NN-radius can be accessed efficiently during verification.

Designing a single index that satisfies all these requirements is non-trivial. For example, the HNSW graph is highly effective for nearest-neighbor retrieval but does not maintain ranked neighbor lists. In contrast, a ranked KNN graph provides ranked neighbor information but is inefficient for query-time nearest-neighbor search. Moreover, neither structure directly supports efficient access to the RkNN sets of data vectors. To address these challenges, we propose HRNN, a hybrid index that integrates an HNSW graph, a ranked KNN graph, and reverse-neighbor lists.

Definition 4.1 (HRNN Index). Given a dataset D and a user-defined parameter $\mathbf{K}$, the HRNN index is defined as $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$, where:

- (1) G_{HNSW} is an HNSW graph constructed over D (Definition 2.8);
- (2) G_{KNN} is a ranked KNN graph constructed over D with parameter $\mathbf{K}$ (Definition 2.6);
- (3) $\mathbf{R}$ is the reverse-neighbor lists derived from G_{KNN} . For each vector $o \in D$, its reverse-neighbor list $\mathbf{R}[o]$ stores all pairs (v, j) such that o is the j -th nearest neighbor of v in G_{KNN} (Definition 2.7).

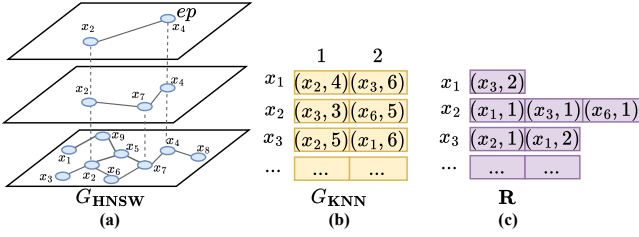


Figure 7: An example of the HRNN index structure.

Example 4.2. Fig. 7 shows the components of HRNN. Fig. 7(a) shows the HNSW graph. Fig. 7(b) presents the ranked KNN graph, where each node o maintains a list of K -nearest neighbors ($K = 2$), and each entry is denoted as $(v_i, \delta(o, v_i))$. For instance, x_2 is the 1NN of x_3 with $\delta(x_3, x_2) = 5$. Fig. 7(c) shows the reverse-neighbor lists derived from G_{KNN} . For example, since x_3 is the 1NN of x_2 and the 2NN of x_1 , its reverse-neighbor list is $R(x_3) = \{(x_2, 1), (x_1, 2)\}$, which records these reverse-neighbor relationships.

THEOREM 4.3. *The HRNN index requires $O(|D| \cdot M \cdot L + |D| \cdot K)$ space, where L is the number of HNSW layers, M is the maximum degree of the HNSW graph, and K is the number of ranked neighbors materialized per point.*

PROOF. The HRNN index consists of three components. First, the HNSW graph stores at most M outgoing neighbors for each vector at each layer. Since there are at most $|D|$ vectors and L layers, the space required by the HNSW graph is bounded by $O(|D| \cdot M \cdot L)$. Second, the ranked KNN graph stores K ranked neighbors for each vector, requiring $O(|D| \cdot K)$ space. Third, the reverse-neighbor lists are obtained by transposing the ranked KNN graph. Each ranked neighbor entry generates exactly one reverse posting, yielding $|D| \cdot K$ reverse postings in total and therefore requiring $O(|D| \cdot K)$ space. Summing the three components gives $O(|D| \cdot M \cdot L + 2|D| \cdot K) = O(|D| \cdot M \cdot L + |D| \cdot K)$. $\square$

4.2 Query Processing

We now describe how to answer an approximate RkNN query for a query vector q using the proposed HRNN index $\mathcal{I} = (G_{HNSW}, G_{KNN}, R)$. If the query vector q were part of the indexed dataset D , its reverse-neighbor list $R[q]$ would directly reveal the points whose neighborhoods contain q . However, in practice, query vectors are typically not part of the indexed dataset. To address this issue, HRNN first retrieves a set of nearby points as *proxies* and uses their reverse-neighbor lists to approximate the unavailable reverse-neighborhood information of q .

Filter. Let $N_m(q) = \{b_1, \dots, b_m\}$ denote the approximate m -nearest neighbors of q returned by the HNSW graph. Rather than treating these points as RkNN candidates, HRNN uses them to discover candidate points. To avoid scanning excessively large reverse-neighbor lists $R[b]$ of the proxy point $b \in N_m(q)$, we introduce a rank threshold Θ and only consider vectors whose ranks do not exceed Θ . This design is based on the intuition that if the rank of an object v with respect to a proxy b exceeds Θ , then b —and thus the query—is unlikely to be within the top- Θ nearest neighbors of v . Hence, v can be safely pruned, as it is unlikely to be an RkNN of q . Formally, the

Algorithm 3: Approximate RkNN Search

Input: Index $\mathcal{I} = (G_{HNSW}, G_{KNN}, R)$, query vector q , target parameter k , proxy size m , rank threshold Θ , HNSW search parameters ep and ef_s

Output: ARkNN result set R

```

1  $R \leftarrow \emptyset, C \leftarrow \emptyset;$ 
2  $N_m(q) \leftarrow \text{GRAPH-SEARCH}(G_{HNSW}, q, m, ep, ef_s);$ 
3 for  $b \in N_m(q)$  do
4   for  $(v, j) \in R(b)$  in ascending order of  $j$  do
5     if  $j > \Theta$  then break;
6      $C \leftarrow C \cup \{v\};$ 
7 for  $o \in C$  do
8    $v_k \leftarrow G_{KNN}[o, k];$ 
9    $\hat{r}_k(o) \leftarrow \delta(o, v_k);$ 
10  if  $\delta(q, o) \leq \hat{r}_k(o)$  then  $R \leftarrow R \cup \{o\};$ 
11 return  $R;$ 
```

candidate set of query q is defined as

$$C_{m, \Theta}(q) = \bigcup_{b \in N_m(q)} \{v \mid (v, j) \in R[b], j \leq \Theta\}. \quad (1)$$

A larger Θ increases candidate coverage and potentially improves recall, but incurs higher candidate-generation and verification costs. Note that K is the number of neighbors materialized in a ranked KNN graph, while $\Theta \leq K$ is a query-time parameter controlling the number of entries scanned from each reverse-neighbor list.

Verification. After candidate generation, HRNN verifies whether each candidate $o \in C_{m, \Theta}(q)$ belongs to the final RkNN result set of q . Existing methods typically compute the k NN-radius of each candidate online, which incurs substantial overhead. In contrast, HRNN leverages the pre-materialized ranked KNN graph G_{KNN} . Since the graph stores the top- K nearest neighbors of every vertex and $K \geq k$, the k -th nearest neighbor of o can be directly retrieved from $G_{KNN}[o]$. Let v_k denote this neighbor. The corresponding distance $\hat{r}_k(o) = \delta(o, v_k)$ serves as an estimate of the k NN-radius of o . Using this pre-materialized radius, a candidate o is accepted if $\delta(q, o) \leq \hat{r}_k(o)$. This verification process reduces candidate validation to a single distance comparison.

Algorithm. Algorithm 3 presents the procedure for answering an approximate RkNN query for a query vector q using the hybrid index $\mathcal{I} = (G_{HNSW}, G_{KNN}, R)$. The algorithm first performs an approximate m -nearest-neighbor search on G_{HNSW} to retrieve a set of proxy vertices $N_m(q)$ that are close to q (Line 2). The details of the graph search procedure are provided in Algorithm 2. For each proxy point $b \in N_m(q)$, the algorithm scans its reverse-neighbor list $R(b)$ in ascending rank order (Lines 3–4). Only vertices whose ranks do not exceed the threshold Θ are inserted into the candidate set C (Lines 5–6). This proxy-guided reverse lookup generates candidate RkNN results without expensive neighborhood expansion. The algorithm then verifies each candidate $o \in C$ (Line 7). Specifically, it retrieves the k -th nearest neighbor v_k of o from G_{KNN} and obtains the corresponding estimated k NN-radius $\hat{r}_k(o) = \delta(o, v_k)$ (Lines 8–9). A candidate is accepted if $\delta(q, o) \leq \hat{r}_k(o)$ (Line 10). All verified candidates are added to the result set R , which is returned as the final RkNN result (Line 11).

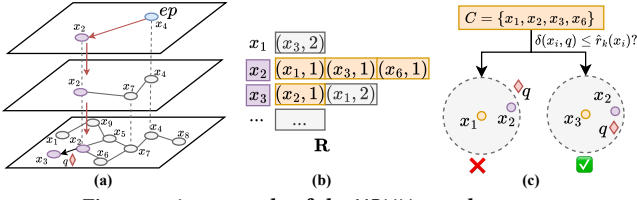


Figure 8: An example of the HRNN search process.

Example 4.4. Fig. 8 illustrates HRNN query processing for a query vector q , with $m = 2$ and $\Theta = 1$. The search first traverses G_{HNSW} and retrieves x_2 and x_3 as proxy vertices, as shown in Fig. 8(a). As illustrated in Fig. 8(b), HRNN then scans the reverse-neighbor lists of the proxies in ascending rank order and inserts vertices whose rank does not exceed Θ into the candidate set C . In this example, x_1 , x_2 , x_3 , and x_6 are added to C . Finally, each candidate is verified by checking whether q falls within its $k\text{NN}$ -radius, which is obtained directly from G_{KNN} . Candidates satisfying this condition (e.g., x_3) are returned as $Rk\text{NN}$ results.

THEOREM 4.5. *Given a query vector q , Algorithm 3 answers the $Rk\text{NN}$ query in expected time $O(\log |D| + m + s(q) + d \cdot u(q))$, where m is the proxy size, $s(q)$ is the number of scanned reverse-neighbor entries, and $u(q)$ is the number of distinct candidates after deduplication (i.e., the size of $C_{m,\Theta}(q)$ in Equation 1).*

PROOF. Proxy retrieval over the HNSW graph requires $O(\log |D| + m)$ expected time. Candidate generation scans $s(q)$ reverse-neighbor entries and therefore takes $O(s(q))$ time. After deduplication, the algorithm verifies $u(q)$ candidates. Each verification consists of one exact distance computation and one constant-time radius lookup, requiring $O(d)$ time. Thus, verification costs $O(d \cdot u(q))$. Summing the three terms yields the stated bound. $\square$

4.3 Index Construction

Constructing the HRNN index $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$ is challenging because it consists of three coupled structures: the HNSW graph, the ranked KNN graph, and the reverse-neighbor lists. A straightforward approach is to construct them independently, but this would incur redundant neighborhood exploration and repeated nearest-neighbor computations.

To avoid this overhead, HRNN adopts a unified construction pipeline. It first builds the HNSW graph and records the neighbors discovered during the bottom-layer searches for each inserted vector. Since these neighbors already approximate the local neighborhood of the vector, they provide high-quality seeds for initializing its ranked KNN list. HRNN then applies NNDescent to refine these initial neighborhoods and construct a high-quality ranked KNN graph. Compared with random initialization, this strategy improves convergence and reduces construction cost. Finally, the reverse-neighbor lists are derived by transposing the ranked KNN graph: if a vector u is the j -th nearest neighbor of a vector v , the pair (v, j) is inserted into the reverse-neighbor list of u . In this way, HRNN constructs all three index structures efficiently while maximizing the reuse of neighborhood information across different index views.

Algorithm. Algorithm 4 constructs the HRNN index $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$ in three phases. First, HRNN builds the HNSW graph G_{HNSW} over the dataset D (Line 1). During HNSW construction, each point is inserted incrementally. For a newly inserted point

Algorithm 4: HRNN Index Construction

Input: Dataset $D \subset \mathbb{R}^d$; HNSW parameters M , ef_c , and m_L ; ranked KNN size K ; NNDescent iterations T

Output: Index $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$

// Phase 1: Build the HNSW graph

- 1 $(G_{\text{HNSW}}, W) \leftarrow \text{BUILD-HNSW}(D, M, ef_c, m_L)$;
- // Phase 2: Construct the ranked KNN graph
- 2 Initialize $G_{\text{KNN}}[o]$ using $W[o]$ for each $o \in D$;
- 3 Execute Lines 2–11 of Algorithm 1 to build G_{KNN} ;
- // Phase 3: Construct reverse-neighbor lists
- 4 Initialize $\mathbf{R}(o) \leftarrow \emptyset$ for all $o \in D$;
- 5 **for each** $o \in D$ **do**
- 6 **for** $j \leftarrow 1$ **to** K **do**
- 7 $v \leftarrow G_{\text{KNN}}[o, j]$;
- 8 Append (o, j) to $\mathbf{R}(v)$;
- 9 **for each** $o \in D$ **do**
- 10 Sort $\mathbf{R}(o)$ in ascending order of rank j ;
- 11 **return** $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$;

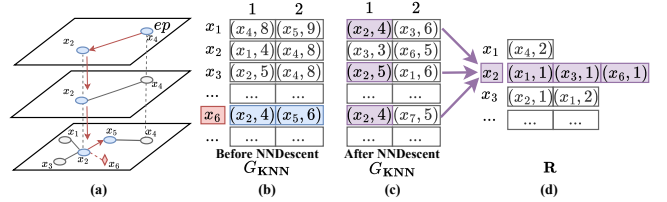


Figure 9: An example of HRNN index construction.

o , HNSW first performs graph search from the current entry point to retrieve a set of approximate nearest neighbors using Algorithm 2. These searched neighbors are then used to connect o to existing vertices in different HNSW layers. Meanwhile, HRNN records the bottom-layer search results as $W[o]$, which provides high-quality initial neighbor candidates for constructing the ranked KNN graph. Second, HRNN constructs the ranked KNN graph G_{KNN} (Lines 2–3). Instead of randomly initializing the neighbor list of each vertex, HRNN initializes $G_{\text{KNN}}[o]$ using $W[o]$, i.e., the candidate neighbors obtained during HNSW construction. Then, Lines 2–11 of Algorithm 1 are executed to refine these initial lists via NNDescent to build a G_{KNN} . Third, HRNN builds the reverse-neighbor lists $\mathbf{R}$ from G_{KNN} (Lines 4). For each vertex o and each rank position j (Line 5–6), the algorithm obtains the j -th nearest neighbor $v = G_{\text{KNN}}[o, j]$ (Line 7) and appends (o, j) to $\mathbf{R}(v)$ (Line 8). Finally, each reverse-neighbor list is sorted by rank j (Line 9–10).

Example 4.6. Fig. 9 illustrates the construction process of the HRNN index. Fig. 9(a) shows the search performed when inserting a new point x_6 into G_{HNSW} , and x_6 is connected to x_2 after pruning the search results. Other points are inserted similarly. The search result $[x_2, x_3]$ is recorded and used to initialize the ranked KNN graph with $K = 2$, as shown in Fig. 9(b). Fig. 9(c) shows how NNDescent refines the ranked KNN graph. For example, in x_6 's neighbor list, the initial 2NN x_3 is replaced by the closer neighbor x_7 . Finally, Fig. 9(d) constructs the reverse-neighbor lists by scanning the refined ranked KNN graph. For instance, since x_2 appears as the 1NN of x_1 , x_3 , and x_6 , its reverse-neighbor list is $\mathbf{R}(x_2) = [(x_1, 1), (x_3, 1), (x_6, 1)]$.

THEOREM 4.7. *Algorithm 4 constructs the HRNN index in expected time $O(|D| \cdot \log |D| + T \cdot |D| \cdot K^2 \cdot d + |D| \cdot K)$, where d is the vector dimension, K is the ranked neighbor-list size, and T is the number of NNDescent refinement iterations.*

PROOF. Algorithm 4 consists of three phases. First, the algorithm builds the HNSW graph over D . HNSW inserts vectors incrementally. For each inserted vector, it performs graph search to locate candidate neighbors and then selects at most M neighbors for graph construction. Following the standard logarithmic search behavior of HNSW, the expected cost of inserting one vector is $O(\log |D|)$. Therefore, inserting all $|D|$ vectors costs $O(|D| \cdot \log |D|)$. During this phase, HNSW also records the bottom-layer search results as initialization seeds for the ranked KNN graph. This recording is performed during HNSW construction and does not change the asymptotic cost.

Second, HRNN constructs the ranked KNN graph. Each vector maintains a ranked list of K neighbors. The initial lists are obtained from the HNSW search results and are refined using NNDescent. In each refinement iteration, for each vector, NNDescent compares candidate pairs generated from its forward and reverse neighbor lists. Since each list has size at most K , the number of candidate comparisons per vector is $O(K^2)$. Each comparison requires computing the distance between two d -dimensional vectors, which costs $O(d)$. Hence, one NNDescent iteration costs $O(|D| \cdot K^2 \cdot d)$. After T iterations, the total refinement cost is $O(T \cdot |D| \cdot K^2 \cdot d)$. Third, HRNN constructs the reverse-neighbor lists by transposing the ranked KNN graph. Since each of the $|D|$ vectors stores K ranked neighbors, the ranked KNN graph contains exactly $|D| \cdot K$ directed neighbor entries. For each entry, HRNN inserts one corresponding reverse posting. Thus, materializing the reverse-neighbor lists costs $O(|D| \cdot K)$. Summing the costs of the three phases gives the result. $\square$

4.4 Index Maintenance

Real-world vector datasets are rarely static. New vectors are continuously generated in applications such as retrieval-augmented generation and recommendation systems. Rebuilding the entire HRNN index after each insertion is prohibitively expensive. Given a newly inserted point o_{new} , maintenance must synchronize three coupled structures. First, o_{new} must be incorporated into the HNSW graph to support future proxy retrieval. Second, if o_{new} becomes one of the top- K nearest neighbors of existing vectors, the corresponding ranked neighbor lists in G_{KNN} must be updated. Third, all affected reverse-neighbor lists must be refreshed to remain consistent with the updated ranked KNN graph. Thus, we first insert o_{new} into G_{HNSW} and then describe how to minimize the maintenance cost of updating G_{KNN} and $\mathbf{R}$.

Exact Affected Area. The key challenge is identifying the existing points in dataset D affected by the insertion of o_{new} . Let $r_K(o)$ denote the distance from o to its current K -th nearest neighbor in G_{KNN} . A vector x is affected if o_{new} should appear in its top- K neighbor list after insertion. Formally, the exact affected set is

$$S^*(o_{\text{new}}) = \{x \in D \mid \delta(x, o_{\text{new}}) < r_K(x)\}. \quad (2)$$

For every point $x \in S^*(o_{\text{new}})$, inserting o_{new} modifies the ranked neighbor list of x , which subsequently triggers updates to reverse-neighbor lists. However, computing $S^*(o_{\text{new}})$ exactly requires evaluating $\delta(x, o_{\text{new}})$ for every point in the dataset, resulting in an $O(|D|)$ scan per insertion. Such a cost is impractical for large-scale vector collections.

Approximate Affected Area. To avoid exhaustive scans, HRNN adopts the same proxy principle used during query processing. Intuitively, points close to o_{new} tend to share similar reverse-neighbor relationships and therefore provide effective hints about the affected region. During HNSW insertion, HRNN obtains a set of nearby proxy points $N_{m_u}(o_{\text{new}})$ and records the search result $W(o_{\text{new}})$ used to initialize the ranked neighbor list of o_{new} . Given an update-time rank threshold $\Theta_u \leq K$, HRNN constructs an approximate affected set by scanning the reverse-neighbor lists of these proxies:

$$\hat{S}_u(o_{\text{new}}) = \bigcup_{b \in N_{m_u}(o_{\text{new}})} \{x \mid (x, j) \in \mathbf{R}(b), j \leq \Theta_u\}. \quad (3)$$

The parameter Θ_u controls the trade-off between maintenance cost and update quality. Larger values increase the coverage of the affected region but require scanning more reverse-neighbor entries.

Maintenance Procedure. After identifying the approximate affected set, HRNN incrementally updates all index components. Specifically, o_{new} is first inserted into the HNSW graph, and the collected search result $W(o_{\text{new}})$ is used to initialize its ranked neighbor list in G_{KNN} . Next, HRNN examines each point $x \in \hat{S}_u(o_{\text{new}})$. If $\delta(x, o_{\text{new}})$ is smaller than the distance to the current K -th neighbor of x , the new vector is inserted into $G_{\text{KNN}}[x]$. If the list already contains K entries, the current farthest neighbor is evicted. Whenever a neighbor list changes, the corresponding reverse-neighbor lists are updated by removing obsolete entries, inserting new entries, and adjusting the ranks of shifted neighbors. Since the affected area is identified approximately rather than exactly, the maintained index may differ slightly from one obtained through full batch reconstruction. However, as demonstrated in Section 5, this discrepancy is negligible in practice. Periodic reconstruction can be performed when necessary to restore the index to its batch-built state.

Algorithm. Algorithm 5 incrementally maintains the HRNN index after the arrival of a new vector o_{new} . The procedure updates the three coupled structures of HRNN: the HNSW graph G_{HNSW} , the ranked KNN graph G_{KNN} , and the reverse-neighbor lists $\mathbf{R}$. In Phase 1, the algorithm inserts o_{new} into G_{HNSW} and reuses the search result $W(o_{\text{new}})$ produced during HNSW insertion (Lines 1–2). The top- m_u closest vectors in this set are selected as proxy vectors $N_{m_u}(o_{\text{new}})$, which serve as proxies for identifying the affected region. In Phase 2, the algorithm constructs the approximate affected area $\hat{S}_u(o_{\text{new}})$ (Lines 3–8). For each proxy vector b , it scans the reverse-neighbor list $\mathbf{R}(b)$ in ascending rank order and collects the owner vectors whose ranks do not exceed the threshold Θ_u . This step follows the same proxy principle used during query processing and avoids the expensive computation of the exact affected area. In Phase 3, the algorithm initializes the ranked neighbor list of o_{new} using the top- K vectors from $W(o_{\text{new}})$ (Lines 9–14). The resulting neighborhood is inserted into G_{KNN} , and the corresponding reverse-neighbor entries are added to $\mathbf{R}$. Finally, in Phase 4, the algorithm refreshes the neighborhoods of all vectors in $\hat{S}_u(o_{\text{new}})$

Algorithm 5: HRNN Index Insertion Maintenance

Input: Index $\mathcal{I} = (G_{\text{HNSW}}, G_{\text{KNN}}, \mathbf{R})$, new vector o_{new} , update parameters m_u, Θ_u
Output: Updated index $\mathcal{I}$
// Phase 1: Insert into HNSW and collect proxy vectors
1 $(G_{\text{HNSW}}, W(o_{\text{new}})) \leftarrow \text{HNSW-INSERT}(G_{\text{HNSW}}, o_{\text{new}});$
2 $B_{m_u}(o_{\text{new}}) \leftarrow \text{top-}m_u \text{ closest vectors in } W(o_{\text{new}}) \text{ to } o_{\text{new}};$
// Phase 2: Construct the approximate affected area
3 $\hat{S}_u(o_{\text{new}}) \leftarrow \emptyset;$
4 **for each** $b \in B_{m_u}(o_{\text{new}})$ **do**
5 **for each** $(x, j) \in \mathbf{R}(b)$ **in ascending order of** j **do**
6 **if** $j > \Theta_u$ **then break;**
7 $\hat{S}_u(o_{\text{new}}) \leftarrow \hat{S}_u(o_{\text{new}}) \cup \{x\};$
// Phase 3: Initialize the new vector
8 $L_{\text{new}} \leftarrow \text{top-}K \text{ vectors in } W(o_{\text{new}}) \text{ ranked by } \delta(o_{\text{new}}, \cdot);$
9 $G_{\text{KNN}}[o_{\text{new}}] \leftarrow L_{\text{new}};$
10 **for each** $v \in L_{\text{new}}$ **do**
11 $j \leftarrow \text{rank of } v \text{ in } L_{\text{new}};$
12 Insert (o_{new}, j) into $\mathbf{R}(v)$, keeping $\mathbf{R}(v)$ sorted by rank;
// Phase 4: Refresh affected neighborhoods
13 **for each** $x \in \hat{S}_u(o_{\text{new}})$ **do**
14 $L_{\text{old}} \leftarrow G_{\text{KNN}}[x];$
15 $r_K(x) \leftarrow \delta(x, G_{\text{KNN}}[x, K]);$
16 **if** $\delta(x, o_{\text{new}}) < r_K(x)$ **then**
17 $L_{\text{upd}} \leftarrow \text{TopK}(L_{\text{old}} \cup \{o_{\text{new}}\}) \text{ ranked by } \delta(x, \cdot);$
18 $G_{\text{KNN}}[x] \leftarrow L_{\text{upd}};$
19 **for each** $v \in L_{\text{old}} \setminus L_{\text{upd}}$ **do**
20 Remove $(x, \cdot)$ from $\mathbf{R}(v)$;
21 **for each** $v \in L_{\text{upd}}$ **do**
22 $j \leftarrow \text{rank of } v \text{ in } L_{\text{upd}};$
23 Insert or update (x, j) in $\mathbf{R}(v)$, keeping $\mathbf{R}(v)$ sorted by rank;
24 **return** $\mathcal{I};$

(Lines 15–30). For each vector x , it first retrieves the current KNN-radius $r_K(x)$. If $\delta(x, o_{\text{new}}) < r_K(x)$, the new vector belongs to the top- K neighborhood of x . The algorithm therefore inserts o_{new} into $G_{\text{KNN}}[x]$, removes the farthest neighbor if necessary, and constructs the updated ranked neighbor list. It then synchronizes the reverse-neighbor lists by removing obsolete postings and inserting or updating the postings associated with the modified neighborhood. Consequently, the ranked KNN graph and reverse-neighbor lists remain consistent after each insertion.

THEOREM 4.8. *For an insertion of o_{new} , the maintenance cost of HRNN is $O(\log |D| + s_u + d \cdot |\hat{S}_u(o_{\text{new}})| + (a_u + 1) \cdot K)$, where s_u is the number of scanned reverse-neighbor entries, $\hat{S}_u(o_{\text{new}})$ is the approximate affected set, and a_u is the number of existing ranked neighbor lists updated.*

PROOF. The maintenance procedure has four parts. First, HRNN inserts o_{new} into the HNSW graph, and one insertion costs $O(\log |D|)$. Second, HRNN discovers the approximate affected set by scanning the rank-truncated reverse-neighbor lists of the update proxies. Let s_u denote the number of scanned reverse-neighbor

Table 3: Dataset statistics.

Dataset	Dimension	# Base	# Query	Domain
SIFT	128	1.0M	10,000	Images
Msong	420	1.0M	1,000	Audio
GIST	960	1.0M	1,000	Images
MSMARCO	1024	1.0M	1,000	Text Embeddings

entries. This step costs $O(s_u)$. Third, for each vector $x \in \hat{S}_u(o_{\text{new}})$, HRNN checks whether o_{new} enters the top- K neighbor list of x . Each check requires one exact distance computation $\delta(x, o_{\text{new}})$ over d dimensions and one comparison with the current K -th neighbor distance. Thus, this step costs $O(d \cdot |\hat{S}_u(o_{\text{new}})|)$. Finally, suppose a_u existing ranked neighbor lists are updated. Refreshing one changed list may shift at most K ranked positions and therefore touches $O(K)$ reverse-neighbor entries. In addition, the ranked neighbor list of o_{new} is initialized and materialized, which costs $O(K)$. Hence, the synchronization cost is $O((a_u + 1) \cdot K)$. Summing the four terms gives the total cost. $\square$

Remark. Building an HRNN index solely through incremental insertions is feasible, but incurs the cumulative cost of maintaining the index after each update. Unlike batch construction in Algorithm 4, which amortizes neighbor refinement through a one-time NNDescent procedure, insertion-based construction performs affected-area discovery and reverse-neighbor-list synchronization for each arriving point. Consequently, its total cost depends on the distribution of update proxies, the sizes of the affected areas, and the number of modified reverse-neighbor lists. As shown in our experiments, insertion-only construction is slower than batch construction on static datasets. However, it avoids expensive index rebuilding and supports continuous updates while preserving the same index structure and query processing algorithm.

5 EXPERIMENT

5.1 Experimental Setup

Datasets. We evaluate on four real-world vector datasets: SIFT [26], Msong [34], GIST [24], and MSMARCO [35]. SIFT, Msong, and GIST are standard benchmarks widely used in prior approximate RkNN studies [41], while MSMARCO represents modern text-embedding retrieval workloads. Table 3 summarizes the dataset statistics. Due to the $O(|D|^2)$ cost of generating exact RkNN ground truth, following prior work [41], our main experiments use datasets with 1 million vectors. To evaluate scalability, we further evaluate HRNN on datasets containing up to 10 million vectors in Exp-8.

Compared Methods. We compare HRNN against representative graph-based approximate RkNN methods, including SFT, RDT, and the state-of-the-art HAMG [41]. Following HAMG, we implement SFT and RDT on top of HNSW to provide strong high-dimensional baselines. We do not compare against spatial-partitioning methods [42, 43, 51] or tree-based RkNN methods [1, 52], as prior studies have shown them to be prohibitively expensive on million-scale high-dimensional datasets.

- HAMG. The state-of-the-art ARkNN method [41]. Since the original Java implementation is not publicly available and the paper does not fully specify all dataset-specific parameter settings, we

re-implement HAMG in C++ based on the published algorithm description. We tune both construction and query parameters over the ranges reported in [41], as well as larger budgets, and report the best recall–latency trade-off achieved on our platform.

- **HNSW-SFT.** The SFT-style baseline used in HAMG [39, 41]. It first retrieves an expanded set of k' nearest neighbors of the query using HNSW and then verifies each candidate for RkNN membership.
- **HNSW-RDT.** The RDT baseline [6, 41] implemented on top of HNSW, following the design adopted in HAMG. It incrementally explores unchecked nearest neighbors around the query and applies the pruning and termination rules of RDT.

Metrics. We evaluate both efficiency and accuracy. Query performance is measured by average queries per second (QPS) over all queries, while accuracy is measured by Recall@ k . For index construction, we report construction time and index size. A construction process is terminated if its runtime exceeds 10^4 seconds.

Implementation. We implement all methods in C++ and compile them using GCC 13.1.0. Experiments are conducted on a server equipped with an Intel Xeon Platinum 8352V CPU @ 2.10GHz and 512 GB RAM running Ubuntu 20.04 LTS. Following common practice in vector search studies [14, 23, 33, 41, 49], all query experiments are executed using a single thread, while index construction uses 64 threads. For HRNN, the HNSW graph is constructed with $M = 16$ and $ef_c = 400$. For the baselines, we tune parameters according to the ranges reported in their original papers and use $C = 8000$ and $d_m = 80$ for HAMG, and $ef_c = 400$ for HNSW-RDT and HNSW-SFT.

Parameter Settings. Unless otherwise specified, all experiments evaluate ARkNN search with $k = 10$. For HRNN, we construct a single index per dataset with $K = 500$. HRNN has two query-time parameters: the number of proxies m and the reverse-neighbor threshold Θ . To generate the recall–throughput trade-off curves in Exp-1, we evaluate $m \in \{1, 3, 5, 10, 20, 30, 50, 100, 150, 200\}$ and $\Theta \in \{10, 20, 30, 50, 100, 200, 300, 500\}$. Each reported HRNN point corresponds to a specific (m, Θ) configuration. For experiments requiring a single configuration at a target recall threshold τ , we select the configuration that achieves the highest throughput among all evaluated settings satisfying Recall@ $k \geq \tau$.

5.2 Experimental Results

Exp-1: Query Performance Comparison. We first compare the end-to-end recall–throughput trade-off of all methods. Fig. 10 reports the results of HRNN, HAMG, HNSW-SFT, and HNSW-RDT on the four datasets. On MSMARCO, at Recall@10 above 0.99, HRNN achieves 185 QPS, outperforming HAMG, HNSW-SFT, and HNSW-RDT by 83.1 $\times$, 10.3 $\times$, and 23.3 $\times$, respectively. On GIST, HAMG achieves only 0.714 recall. Although HNSW-SFT and HNSW-RDT can further improve recall by expanding the query-centered search region, their throughput drops to roughly 1 QPS or below. Specifically, HNSW-SFT reaches 0.958 recall at only 0.372 QPS, while HNSW-RDT reaches 0.880 recall at 1.076 QPS. We therefore terminate further expansion once the throughput becomes prohibitively low. In contrast, HRNN achieves 0.999 recall at 39.3 QPS.

These results expose the limitations of existing methods for high-dimensional RkNN search, especially on datasets with a pronounced mismatch between nearby neighbors and true RkNN results. By using nearby neighbors as proxies instead of direct candidates and verifying only reverse-lookup candidates with pre-materialized kNN-radius information, HRNN substantially reduces both candidate-generation and verification overhead, yielding consistently better recall–throughput trade-offs.

Exp-2: Query Time Breakdown. We analyze how query time is distributed across different stages of HRNN. We decompose each query into three phases: (1) proxy retrieval using the navigation graph G_{HNSW} , (2) reverse-neighbor list scanning for candidate generation, and (3) candidate verification using pre-materialized kNN-radius retrieved from the ranked kNN graph G_{KNN} . For each dataset, we select the lowest-latency configuration from the same (m, Θ) grid used in Exp-1 under Recall@10 thresholds of 0.95 and 0.99.

Fig. 11 shows that proxy retrieval incurs a small and nearly constant cost across datasets and recall targets, ranging from 0.43ms to 1.7ms. Reverse-neighbor list scanning also remains lightweight because HRNN accesses only rank-truncated posting entries. In contrast, candidate verification becomes the dominant cost at high recall levels, as more candidates must be validated. For example, verification accounts for 78.5%–86.3% of the total query time on GIST and increases from 35.7% to 74.3% on MSMARCO as the target recall rises from 0.95 to 0.99. Nevertheless, compared with existing methods (see also Fig. 5), HRNN reduces verification overhead by replacing expensive online kNN-radius computation with direct lookups from the materialized ranked kNN graph, thereby mitigating one of the major bottlenecks of prior RkNN approaches.

Exp-3: Index Construction Time and Space Consumption. We next compare the index construction time and index size of different methods. The results are reported in Table 4 and Table 5, respectively. Table 4 compares the construction cost of graph HNSW, index HRNN, and index HAMG. Compared with the graph HNSW, HRNN incurs additional offline overhead to construct the ranked KNN graph and materialize the reverse-neighbor lists. Nevertheless, it remains substantially cheaper to build than HAMG. Across the four datasets, HAMG requires 5.2–14.4 $\times$ longer construction time than HRNN, with the largest gap observed on MSMARCO.

Table 5 reports the index size of different methods. The *Base* column shows the raw vector storage size as a reference, while the method columns report index overhead only and exclude the raw vectors. HRNN stores both the navigation graph and the reverse-neighbor lists derived from the ranked KNN graph. As a result, it requires more memory than HNSW. This overhead is most pronounced on low-dimensional datasets such as SIFT, where the raw vector storage is relatively small. On higher-dimensional datasets, the overhead becomes more moderate. For example, the total space consumption of HRNN is only 1.54 $\times$ and 1.52 $\times$ the raw vector size on GIST and MSMARCO, respectively.

Exp-4: Parameter Sensitivity. We study the impact of HRNN’s two query-time parameters: the number of proxies m and the reverse-neighbor threshold Θ . Both parameters increase candidate coverage but affect different stages of the search process. Specifically, a larger m expands the set of query-side proxies, while a larger

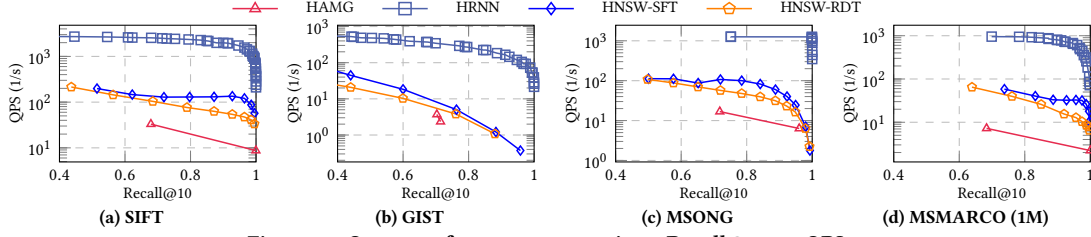


Figure 10: Query performance comparison: Recall@10 vs. QPS.

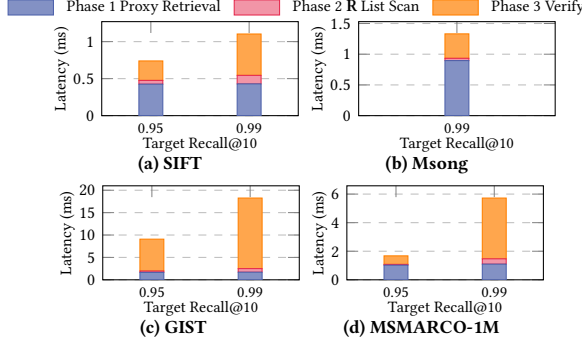


Figure 11: Query time breakdown at recall@10 targets (0.95 and 0.99).

Table 4: Index construction time (seconds).

Dataset	HNSW	HRNN	HAMG
SIFT	49	185	954
Msong	91	280	1537
GIST	229	452	4286
MSMARCO	199	553	7947

Table 5: Index space consumption (MB).

Dataset	Base	HNSW	HAMG	HRNN	HRNN Total/Base
SIFT	512.0	100.7	206.6	2112.6	5.13×
Msong	1670.2	83.3	165.2	2083.4	2.25×
GIST	3840.0	71.8	199.9	2083.8	1.54×
MSMARCO	4096.0	103.4	275.3	2114.9	1.52×

Θ relaxes the rank constraint when scanning reverse-neighbor lists. Since the ranked KNN graph is constructed offline with a fixed size of $K = 500$, all settings satisfying $\Theta \leq K$ are supported by the same index.

Table 6 reports the highest-throughput configurations that satisfy Recall@10 thresholds of 0.95 and 0.99. The optimal parameter settings vary across datasets. For example, Msong saturates with $(m, \Theta) = (3, 10)$, whereas GIST requires larger candidate coverage, reaching Recall@10 ≥ 0.99 only with configurations such as (300, 50). SIFT and MSMARCO exhibit intermediate behavior. These results reflect the degree of mismatch between nearby neighbors and RkNN results, with larger mismatches requiring larger values of (m, Θ) . Importantly, HRNN can adapt to such variations by adjusting (m, Θ) at query time, without rebuilding the index.

Fig. 12 reports the complete (m, Θ) parameter grid evaluated in our experiments. Increasing either m or Θ generally improves Recall@10, but reduces throughput. This trend is consistent across

Table 6: Best HRNN parameter configurations for target recall@10.

Dataset	Recall@10 ≥ 0.95		Recall@10 ≥ 0.99	
	(m, Θ)	Recall / QPS	(m, Θ)	Recall / QPS
GIST	(10, 500)	0.9622 / 105.1	(50, 300)	0.9935 / 53.4
SIFT	(20, 100)	0.9646 / 1561.8	(50, 100)	0.9916 / 1015.5
MSMARCO	(30, 100)	0.9599 / 607.5	(50, 500)	0.9919 / 184.5
Msong	(3, 10)	0.9990 / 1253.1	(3, 10)	0.9990 / 1253.1

all datasets, reflecting the trade-off between candidate coverage and query efficiency.

Exp-5: Ablation Study. We conduct ablation studies to evaluate the impact of the key components of HRNN. Specifically, we examine three variants: (1) replacing G_{HNSW} for proxy retrieval, (2) constructing the ranked G_{KNN} without using the neighbor seeds collected from G_{HNSW} , and (3) performing candidate verification without pre-materialized k NN-radius information.

Navigation graph replacement. We first replace the HNSW graph used for proxy retrieval while keeping all other components unchanged. Fig. 13 compares HNSW and NSG. On GIST, the two graphs achieve similar performance in the high-recall region. At Recall@10 around 0.99, HNSW achieves 0.9906 recall at 63.9 QPS, while NSG achieves 0.9907 recall at 67.9 QPS. On SIFT, both graphs reach comparable recall, but HNSW delivers higher throughput. At Recall@10 around 0.99, HNSW achieves 0.9910 recall at 1441.8 QPS, compared with 0.9917 recall at 1082.5 QPS for NSG. These results indicate that HRNN is not tied to a specific navigation graph, although the choice of graph affects query latency.

KNN graph construction with HNSW-Seeding. We next evaluate how the ranked KNN graph is initialized prior to NNDescent refinement. Fig. 14 compares standard NNDescent with random initialization against HRNN’s HNSW-seeded construction. On GIST, random initialization reaches only 0.9066 KNNG recall after 435 s of refinement. In contrast, reusing HNSW insertion candidates as seeds yields an initial recall of 0.4880 and converges to perfect recall after 310.85 s. A similar trend is observed on SIFT, where the final recall/construction time improves from 0.9874/387.47 s to 1.0/272.51 s. These results demonstrate that the HNSW insertion process provides high-quality initialization for NNDescent, reducing refinement cost while improving the quality of the resulting ranked KNN graph.

kNN-radius materialization. Finally, we evaluate whether HRNN can remove or approximate its materialized radius information. Table 7 reports results on GIST with $m = 20$, $\Theta = 500$, and $k = 10$. The *Gold Radius* variant replaces HRNN’s G_{KNN} -derived radius estimates $\hat{r}_k(x)$ with exact radius $r_k(x)$ computed by brute force. Recall changes only marginally, from 0.9879 to 0.9880, indicating that the ranked KNN graph provides sufficiently accurate radius

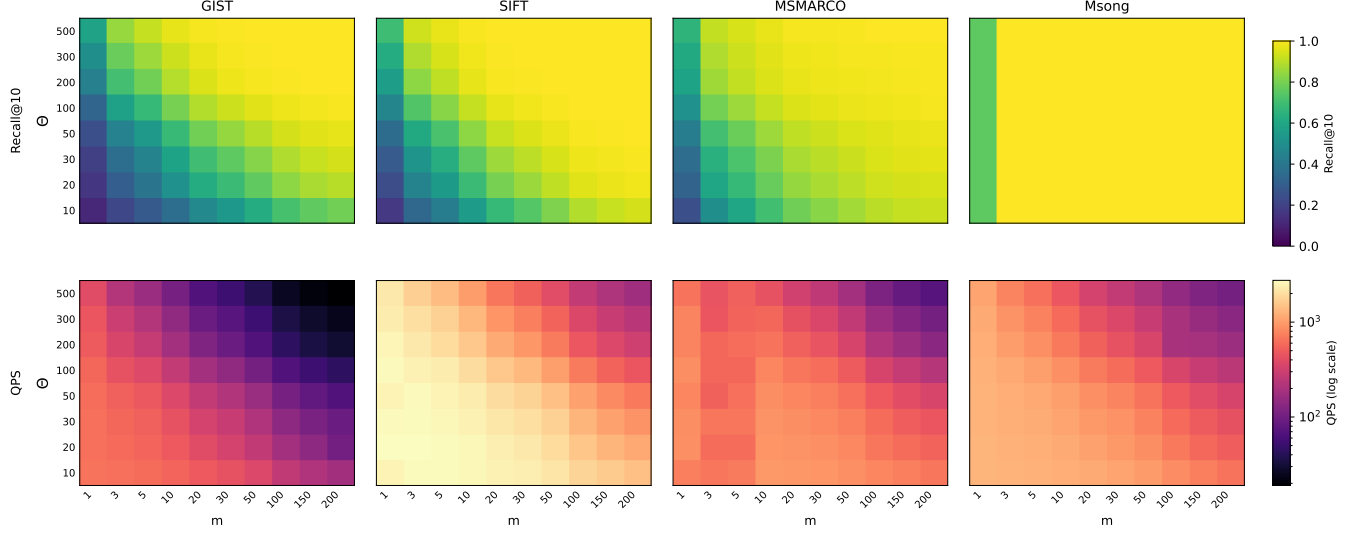


Figure 12: Recall@10 (top) and QPS (bottom, log scale) over the evaluated (m, Θ) parameter grid.

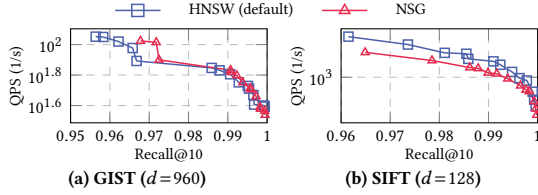


Figure 13: Effect of navigation graph choice (HNSW vs. NSG).

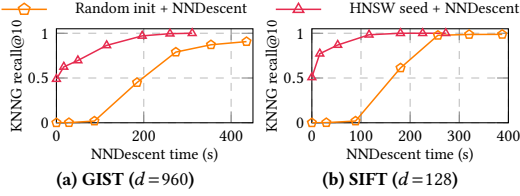


Figure 14: Effect of HNSW-seeding on KNN graph construction.

Table 7: k NN-radius materialization ablation on GIST-1M.

Variant	Recall@10	Latency (ms)	Avg. Results
HRNN	0.9879	14.74	9.89
Gold Radius	0.9880	14.39	9.84
No reverse-neighbor Lists	0.9998	372.04	9.96

estimates for verification. Removing the reverse-neighbor lists (i.e., the variant *No Reverse-neighbor Lists*) forces HRNN to verify all 1M data points, increasing latency from 14.74 ms to 372.04 ms. This confirms that reverse-neighbor lists are essential for selective candidate generation and efficient verification.

Exp-6: Varying k . We evaluate HRNN under standard Rk NN settings with varying values of $k \in 1, 10, 30, 50, 90$. For each dataset and each value of k , we evaluate the same (m, Θ) parameter grid used in Exp-1 and report the highest-throughput configuration satisfying target Recall@ k thresholds of 0.95 and 0.99.

Fig. 15 shows that HRNN maintains stable recall-throughput trade-offs across the entire range of k . At the 0.99 recall target, increasing k from 1 to 90 reduces throughput from 1245 to 400 QPS

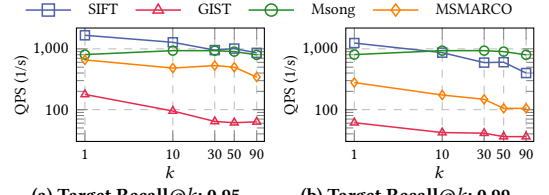


Figure 15: Performance of HRNN across different k values.

on SIFT, from 61 to 36 QPS on GIST, and from 279 to 105 QPS on MSMARCO. On Msong, the Recall@ $k \geq 0.95$ and Recall@ $k \geq 0.99$ curves coincide because the lowest-latency configuration already achieves Recall@ $k \approx 0.999$ for all evaluated values of k , while maintaining a throughput of approximately 800–930 QPS. These results demonstrate that HRNN remains robust across a wide range of target k values. Cross-method comparisons are reported separately in Exp-1 under the common setting of $k = 10$.

Exp-7: Insertion-Based Maintenance. We evaluate insertion-based maintenance by varying the initial batch fraction s : the first $s \cdot n$ points are batch-built and the remaining points are inserted. Thus $s = 1$ is pure batch construction and $s = 0$ is pure insertion construction. For each s , we report the highest-QPS operating point reaching each target Recall@10.

Fig. 16 shows that insertion-based maintenance largely preserves the recall-QPS trade-off. On SIFT, pure insertion reaches 1429, 1266, and 775 QPS at Recall@10 targets 0.90, 0.95, and 0.99, respectively, compared with 1786, 1370, and 813 QPS for pure batch construction (that is, the index is constructed once over the entire dataset). On GIST, pure insertion is comparable to pure batch at the 0.90 and 0.95 targets, reaching 140/89 QPS versus 133/86 QPS. The maintained index thus does not collapse as the insertion fraction grows.

The main overhead is construction time, and this overhead is expected because insertion-based construction synchronizes multiple materialized views after each arriving point. Building entirely through insertions costs 155s versus 86s on SIFT and 760s versus

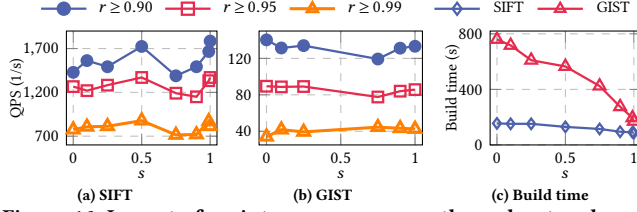


Figure 16: Impact of maintenance on query throughput and construction time. Panels (a)–(b) show throughput at Recall@10 targets of 0.90, 0.95, and 0.99; panel (c) shows total construction time.

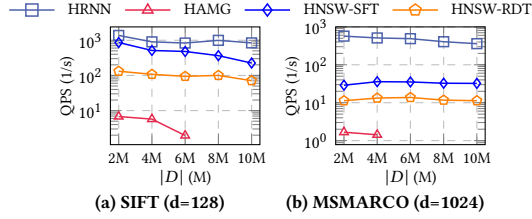


Figure 17: Cross-method scalability at recall@10 ≥ 0.95.

170s on GIST, i.e., 1.80× and 4.47× more than pure batch construction. This cost comes from maintaining the HNSW graph, ranked KNN graph, materialized radius estimates, and reverse-neighbor lists during insertion. Overall, HRNN supports continuous arrivals by paying additional write-side maintenance cost while keeping search efficiency stable.

Exp-8: Scalability to 10M Vectors. We evaluate scalability on the SIFT and MSMARCO datasets with sizes $|D| \in \{2, 4, 6, 8, 10\}$ M. We compare HRNN, HNSW-SFT, and HNSW-RDT across all dataset sizes. For HAMG, we report only completed runs; larger instances are omitted when construction time exceeds 10⁴ seconds and query throughput remains below 10 QPS. Fig.17 compares query throughput at a common target of Recall@10 ≥ 0.95. On SIFT, HRNN is 1.59–3.74× faster than HNSW-SFT and 8.4–11.9× faster than HNSW-RDT across all dataset sizes. The advantage over HAMG is even more pronounced. For example, on SIFT-6M, HRNN achieves 834.3 QPS, whereas HAMG achieves only 1.96 QPS. These results demonstrate the superior scalability of HRNN on large-scale datasets.

Fig. 18 shows that HRNN maintains a stable recall–throughput trade-off up to 10M vectors. At Recall@10 ≥ 0.95, HRNN achieves 842 QPS on SIFT-10M and 360.2 QPS on MSMARCO-10M. It further reaches Recall@10 = 0.9975 on SIFT-10M while sustaining 211.7 QPS.

Fig. 19 reports index construction time as the dataset size increases. On SIFT-10M, HRNN builds the index in 2335 s, compared with 559 s for HNSW. On MSMARCO-10M, HRNN requires 5006 s, compared with 2433 s for HNSW. This additional offline cost stems from constructing the ranked KNN graph and materializing the reverse-neighbor lists. Nevertheless, it remains substantially lower than that of HAMG, which already requires 11405 s on SIFT-6M and 37837 s on MSMARCO-4M.

6 RELATED WORK

As introduced in Section 3, existing methods for (approximate) RkNN search generally follow a filter-and-verification framework.

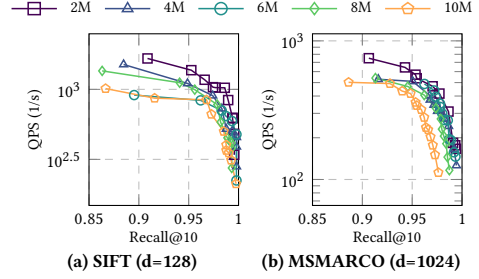


Figure 18: Scalability of HRNN across dataset sizes.

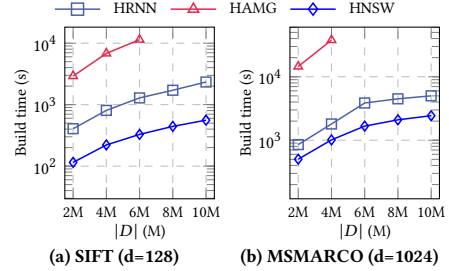


Figure 19: Index construction scalability across dataset sizes.

For most existing methods, the verification stage is similar: they issue a standard kNN search for each candidate to determine whether the candidate belongs to the RkNN result set of the query vector. Therefore, existing methods mainly differ in how they generate the candidate set. Based on this distinction, we categorize existing methods into three groups below.

Classical Methods. Early studies mainly focused on exact RkNN search in low-dimensional vector spaces. One category is partition-based methods [42, 43, 51, 57], which divide the data space into geometric regions such as half-spaces or influence zones. These methods prune impossible candidates based on the spatial partition and the relative position between the query vector and the partitioned regions. Another category is precomputation-based methods [1, 52]. These methods typically use tree-based spatial indexes (e.g., M-trees) and derive conservative distance bounds for subtrees to prune unpromising points during query processing. However, these classical methods suffer severely from the curse of dimensionality. In high-dimensional spaces, partition-based methods lose their discriminative power and often degenerate into scanning large portions of the dataset. Precomputation-based methods also incur high index construction and maintenance costs due to expensive distance-bound computation.

Expansion-Based Methods. To scale RkNN queries to high-dimensional vectors, expansion-based methods such as SFT [39] and RDT [6] were proposed. These methods aim to find approximate RkNN results by progressively exploring nearby neighbors around the query vector q until a stopping condition is satisfied. As analyzed in Section 3, these methods become inefficient when nearby neighbors do not align well with the true RkNN results, since they must explore a large neighborhood region to maintain high search accuracy.

The State-of-the-Art and Beyond. The state-of-the-art method HAMG [41] adapts proximity graphs for RkNN search. HAMG

leverages the key property of the MRN graph that guarantees true RkNN results lie within the k -hop neighborhood of the query vector on the graph. Based on this property, HAMG retrieves nearby graph neighbors as candidates. However, HAMG may still explore a large number of unnecessary candidates because the graph neighborhood can be substantially larger than the true RkNN result set.

Proximity graphs such as HNSW [33] and NSG [14] have become the dominant paradigm for high-dimensional approximate kNN search. HAMG elegantly exploits the properties of the MRN graph, which is also a proximity graph. Since exact construction of an MRN graph is expensive, HAMG heuristically adapts the HNSW graph to better approximate the MRN structure. Although this heuristic adaptation weakens the theoretical guarantees of the original MRN graph, it remains effective in practice. There is a large body of work has studied high-dimensional kNN search from different perspectives, including distance computation optimization [11, 15, 55, 56], novel index structures [16, 44, 48], and search optimization techniques [9, 17, 25, 29, 31]. Since this paper focuses on RkNN search, we omit detailed a discussion of these methods.

7 CONCLUSION

This paper studies approximate RkNN search over high-dimensional vectors. Motivated by the limitations of existing methods, we propose HRNN, a hybrid graph index that treats nearby neighbors as proxy points and materializes kNN-radius offline to avoid costly online computation. HRNN integrates a navigation graph, a ranked KNN graph, and reverse-neighbor lists, together with efficient index construction and append-only maintenance algorithms. Experiments on real-world datasets demonstrate that HRNN consistently outperforms existing methods in both efficiency and scalability while maintaining high search accuracy.

REFERENCES

- [1] Elke Ahtert, Christian Böhm, Peer Kröger, Peter Kunath, Alexey Pryakhin, and Matthias Renz. 2006. Efficient reverse k-nearest neighbor search in arbitrary metric spaces. In *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data* (Chicago, IL, USA) (SIGMOD '06). Association for Computing Machinery, New York, NY, USA, 515–526. <https://doi.org/10.1145/1142473.1142531>
- [2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=hSyW5go0v8>
- [3] Ilias Azizi, Karima Echihabi, and Themis Palpanas. 2023. Elpis: Graph-based similarity search for scalable data science. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1548–1559.
- [4] Gautam Bhattacharya, Koushik Ghosh, and Ananda S. Chowdhury. 2015. Outlier detection using neighborhood rank difference. *Pattern Recogn. Lett.* 60, C (Aug. 2015), 24–31. <https://doi.org/10.1016/j.patrec.2015.04.004>
- [5] Avory Bryant and Krzysztof Cios. 2018. RNN-DBSCAN: A Density-Based Clustering Algorithm Using Reverse Nearest Neighbor Density Estimates. *IEEE Transactions on Knowledge and Data Engineering* 30, 6 (2018), 1109–1121. <https://doi.org/10.1109/TKDE.2017.2787640>
- [6] Guillaume Casanova, Elias Englmeier, Michael E. Houle, Peer Kröger, Michael Nett, Erich Schubert, and Arthur Zimek. 2017. Dimensional testing for reverse k-nearest neighbor search. *Proc. VLDB Endow.* 10, 7 (March 2017), 769–780. <https://doi.org/10.14778/3067421.3067426>
- [7] Muhammad Aamir Cheema, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2011. Influence zone: Efficiently processing reverse k nearest neighbors queries. In *Proceedings of the 2011 IEEE 27th International Conference on Data Engineering (ICDE '11)*. IEEE Computer Society, USA, 577–588. <https://doi.org/10.1109/ICDE.2011.5767904>
- [8] Meng Chen, Kai Zhang, Zhenying He, Yanan Jing, and X. Sean Wang. 2024. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 17, 11 (July 2024), 2735–2749. <https://doi.org/10.14778/3681954.3681959>
- [9] Patrick Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. 2023. FINGER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference 2023* (Austin, TX, USA) (WWW '23). Association for Computing Machinery, New York, NY, USA, 3225–3235. <https://doi.org/10.1145/3543507.3583318>
- [10] Qi-Zhu Dai, Zhong-Yang Xiong, Jiang Xie, Xiao-Xia Wang, Yu-Fang Zhang, and Jia-Xing Shang. 2019. A novel clustering algorithm based on the natural reverse nearest neighbor structure. *Inf. Syst.* 84, C (Sept. 2019), 1–16. <https://doi.org/10.1016/j.is.2019.04.001>
- [11] Liwei Deng, Penghao Chen, Ximu Zeng, Tianfu Wang, Yan Zhao, and Kai Zheng. 2024. Efficient Data-Aware Distance Comparison Operations for High-Dimensional Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 3 (Nov. 2024), 812–821. <https://doi.org/10.14778/3712221.3712244>
- [12] Wei Dong, Charikar Moses, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th International Conference on World Wide Web* (Hyderabad, India) (WWW '11). Association for Computing Machinery, New York, NY, USA, 577–586. <https://doi.org/10.1145/1963405.1963487>
- [13] Cong Fu, Changxu Wang, and Deng Cai. 2022. High Dimensional Similarity Search With Satellite System Graph: Efficiency, Scalability, and Unindexed Query Compatibility. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 8 (2022), 4139–4150. <https://doi.org/10.1109/TPAMI.2021.3067706>
- [14] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.* 12, 5 (Jan. 2019), 461–474. <https://doi.org/10.14778/3303753.3303754>
- [15] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proc. ACM Manag. Data* 1, 2, Article 137 (June 2023), 27 pages. <https://doi.org/10.1145/3589282>
- [16] Yutong Gou, Jianyang Gao, Yuxuan Xu, and Cheng Long. 2025. SymphonyQG: Towards Symphonious Integration of Quantization and Graph for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 1, Article 80 (Feb. 2025), 26 pages. <https://doi.org/10.1145/3709730>
- [17] Hao Guo and Youyou Lu. 2026. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. *ACM Trans. Storage* 22, 2, Article 12 (March 2026), 30 pages. <https://doi.org/10.1145/3793926>
- [18] Kiana Hajebi, Yasin Abbasi-Yadkori, Hossein Shahbazi, and Hong Zhang. 2011. Fast approximate nearest-neighbor search with k-nearest neighbor graph. In *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence - Volume Volume Two* (Barcelona, Catalonia, Spain) (IJCAI'11). AAAI Press, 1312–1317.
- [19] Reza Heydari-Gharaei, Rasoul Sharifi, Shima Kashef, and Hossein Nezamabadi-pour. 2025. A robust approach for outlier detection based on the ratio of number of reverse neighbors to neighbors. *Pattern Anal. Appl.* 28, 1 (Jan. 2025), 30. <https://doi.org/10.1007/s10044-024-01372-y>
- [20] Lihua Hu, Hongkai Liu, Jifu Zhang, and Aiqin Liu. 2022. KR-DBSCAN: A density-based clustering algorithm based on reverse nearest neighbor and influence space. *Expert Syst. Appl.* 186, C (Dec. 2022), 8. <https://doi.org/10.1016/j.eswa.2021.115763>
- [21] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 604–613.
- [22] Piotr Indyk and Haike Xu. 2023. Worst-case performance of popular approximate nearest neighbor search implementations: guarantees and limitations. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 2891, 18 pages.
- [23] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf
- [24] Hervé Jegou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (Jan. 2011), 117–128. <https://doi.org/10.1109/TPAMI.2010.57>
- [25] Sungjun Jung, Yongsang Park, Haeun Lee, Young H. Oh, and Jae W. Lee. 2025. Angular Distance-Guided Neighbor Selection for Graph-Based Approximate Nearest Neighbor Search. In *Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) (WWW '25). Association for Computing Machinery, New York, NY, USA, 4014–4023. <https://doi.org/10.1145/3696410.3714870>
- [26] Hervé Jegou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in one billion vectors: Re-rank with source coding. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 861–864. <https://doi.org/10.1109/ICASSP.2011.5946540>
- [27] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

- [28] Binhong Li, Xiao Yan, and Shangqi Lu. 2026. Fast-Convergent Proximity Graphs for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 4, 1, Article 36 (April 2026), 24 pages. <https://doi.org/10.1145/3786650>
- [29] Liang Li, Shufeng Gong, Yanan Yang, Yiduo Wang, and Jie Wu. 2026. I/O Optimizations for Graph-Based Disk-Resident Approximate Nearest Neighbor Search: A Design Space Exploration. <https://doi.org/10.48550/arXiv.2602.21514> [cs]
- [30] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. 2021. HVS: hierarchical graph structure based on voronoi diagrams for solving approximate nearest neighbor search. *Proceedings of the VLDB Endowment* 15, 2 (2021), 246–258.
- [31] Kejing Lu, Chuan Xiao, and Yoshiharu Ishikawa. 2024. Probabilistic routing for graph-based approximate nearest neighbor search. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 1347, 19 pages.
- [32] Shangqi Lu and Yufei Tao. 2025. Proximity Graphs for Similarity Search: Fast Construction, Lower Bounds, and Euclidean Separation. *Proc. ACM Manag. Data* 3, 5, Article 280 (Nov. 2025), 25 pages. <https://doi.org/10.1145/3767716>
- [33] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (April 2020), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- [34] Brian McFee, Thierry Bertin-Mahieux, Daniel P.W. Ellis, and Gert R.G. Lanckriet. 2012. The million song dataset challenge. In *Proceedings of the 21st International Conference on World Wide Web (Lyon, France) (WWW '12 Companion)*. Association for Computing Machinery, New York, NY, USA, 909–916. <https://doi.org/10.1145/2187980.2188222>
- [35] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A Human Generated Machine Reading Comprehension Dataset. In *CoCo@NIPS (CEUR Workshop Proceedings)*, Tarek Richard Besold, Antoine Bordes, Artur S. d'Ávila Garcez, and Greg Wayne (Eds.), Vol. 1773. CEUR-WS.org. <http://dblp.uni-trier.de/db/conf/nips/coco2016.html#NguyenRSGTMD16>
- [36] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient Approximate Nearest Neighbor Search in Multi-dimensional Databases. *Proc. ACM Manag. Data* 1, 1, Article 54 (May 2023), 27 pages. <https://doi.org/10.1145/3588908>
- [37] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. 2015. Reverse Nearest Neighbors in Unsupervised Distance-Based Outlier Detection. *IEEE Transactions on Knowledge and Data Engineering* 27, 5 (2015), 1369–1382. <https://doi.org/10.1109/TKDE.2014.2365790>
- [38] Payel Sadhukhan and Sarbani Palit. 2019. Reverse-nearest neighborhood based oversampling for imbalanced, multi-label datasets. *Pattern Recogn. Lett.* 125, C (July 2019), 813–820. <https://doi.org/10.1016/j.patrec.2019.08.009>
- [39] Amit Singh, Hakan Ferhatosmanoglu, and Ali Şaman Tosun. 2003. High dimensional reverse nearest neighbor queries. In *Proceedings of the Twelfth International Conference on Information and Knowledge Management (New Orleans, LA, USA) (CIKM '03)*. Association for Computing Machinery, New York, NY, USA, 91–98. <https://doi.org/10.1145/956863.956882>
- [40] Anbang Song, Ziqiang Yu, Wei Liu, Yating Xu, and Mingjin Tao. 2025. BRkNN-light: Batch Processing of Reverse k-Nearest Neighbor Queries for Moving Objects on Road Networks. In *Proceedings of the 19th International Symposium on Spatial and Temporal Data (SSTD '25)*. Association for Computing Machinery, New York, NY, USA, 80–89. <https://doi.org/10.1145/3748777.3748791>
- [41] Yitong Song, Kai Wang, Bin Yao, Zhida Chen, Jiong Xie, and Feifei Li. 2024. Efficient Reverse k Approximate Nearest Neighbor Search Over High-Dimensional Vectors. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 4262–4274. <https://doi.org/10.1109/ICDE60146.2024.00325>
- [42] Yufei Tao, Dimitris Papadias, and Xiang Lian. 2004. Reverse kNN search in arbitrary dimensionality. In *Proceedings of the Thirtieth International Conference on Very Large Data Bases - Volume 30 (Toronto, Canada) (VLDB '04)*. VLDB Endowment, 744–755.
- [43] Yufei Tao, Dimitris Papadias, Xiang Lian, and Xiaokui Xiao. 2007. Multidimensional reverse kNN search. *The VLDB Journal* 16, 3 (July 2007), 293–316. <https://doi.org/10.1007/s00778-005-0168-2>
- [44] Kento Tatsuno, Daisuke Miyashita, Taiga Ikeda, Kiyoshi Ishiyama, Kazunari Sumiyoshi, and Jun Deguchi. 2025. AiSAQ: All-in-Storage ANNS with Product Quantization for DRAM-free Information Retrieval. [arXiv:2404.06004](https://arxiv.org/abs/2404.06004) [cs.IR] <https://arxiv.org/abs/2404.06004>
- [45] Hongya Wang, Wenlong Wu, Cong Luo, Aobei Bian, Chunguang Meng, Yishuo Wu, and Ji Sun. 2025. Boosting Accuracy and Efficiency for Vector Retrieval with Local Scaling Graph. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 336–348. <https://doi.org/10.1109/ICDE65448.2025.00032>
- [46] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, and Lu Chen. 2024. An Interactive Multi-Modal Query Answering System with Retrieval-Augmented Large Language Models. *Proc. VLDB Endow.* 17, 12 (Aug. 2024), 4333–4336. <https://doi.org/10.14778/3685800.3685868>
- [47] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *Proc. ACM Manag. Data* 3, 3, Article 123 (June 2025), 29 pages. <https://doi.org/10.1145/3725260>
- [48] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proc. ACM Manag. Data* 2, 1, Article 14 (March 2024), 27 pages. <https://doi.org/10.1145/3639269>
- [49] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 14, 11 (July 2021), 1964–1978. <https://doi.org/10.14778/3476249.3476255>
- [50] Zeyu Wang, Peng Wang, Themis Palpanas, and Wei Wang. 2023. Graph- and Tree-based Indexes for High-dimensional Vector Similarity Search: Analyses, Comparisons, and Future Directions. *IEEE Data Eng. Bull.* 47, 3 (2023), 3–21. <http://sites.computer.org/debull/A23sept/p3.pdf>
- [51] Wei Wu, Fei Yang, Chee-Yong Chan, and Kian-Lee Tan. 2008. Finch: Evaluating reverse k-nearest-neighbor queries on location data. *Proceedings of the VLDB Endowment* 1, 1 (2008), 1056–1067.
- [52] Chenyi Xia, Wynne Hsu, and Mong Li Lee. 2005. ERkNN: efficient reverse k-nearest neighbors retrieval with local kNN-distance estimation. In *Proceedings of the 14th ACM International Conference on Information and Knowledge Management (Bremen, Germany) (CIKM '05)*. Association for Computing Machinery, New York, NY, USA, 533–540. <https://doi.org/10.1145/1099554.1099697>
- [53] Jiadong Xie, Jeffrey Xu Yu, and Yingfan Liu. 2025. Graph based k-nearest neighbor search revisited. *ACM Transactions on Database Systems* 50, 4 (2025), 1–30.
- [54] Congyun Yang and King-Ip Lin. 2001. An index structure for efficient reverse nearest neighbor queries. In *Proceedings 17th International Conference on Data Engineering*. IEEE, 485–492.
- [55] Mingyu Yang, Liuchang Jing, Wentao Li, and Wei Wang. 2026. Quantization Meets Projection: A Happy Marriage for Approximate k-Nearest Neighbor Search. [arXiv:2411.06158](https://arxiv.org/abs/2411.06158) [cs.DB] <https://arxiv.org/abs/2411.06158>
- [56] Mingyu Yang, Wentao Li, Jiabao Jin, Xiaoyao Zhong, Xiangyu Wang, Zhitao Shen, Wei Jia, and Wei Wang. 2025. Effective and General Distance Computation for Approximate Nearest Neighbor Search. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 1098–1110. <https://doi.org/10.1109/ICDE65448.2025.00087>
- [57] Shiyu Yang, Muhammad Aamir Cheema, Xuemin Lin, and Ying Zhang. 2014. SLICE: Reviving regions-based pruning for reverse k nearest neighbors queries. In *2014 IEEE 30th International Conference on Data Engineering*. 760–771. <https://doi.org/10.1109/ICDE.2014.6816698>
- [58] Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. 2024. Making Retrieval-Augmented Language Models Robust to Irrelevant Context. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=ZS4m74kZpH>
- [59] Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. 2024. RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=S1fc92uemC>
- [60] Aimin Zhang, Hualong Yu, Zhangjun Huan, Xibei Yang, Shang Zheng, and Shang Gao. 2022. SMOTE-RkNN: A hybrid re-sampling method based on SMOTE and reverse k-nearest neighbors. *Information Sciences* 595 (2022), 70–88. <https://doi.org/10.1016/j.ins.2022.02.038>